

Slide 1-1-1-1

Origine des Design Patterns avec le Gang of Four

- Solutions éprouvées à des problèmes récurrents en programmation orientée objet
- Inspirés de l'architecture par Christopher Alexander (années 1970)
- Adaptés au logiciel par le **Gang of Four** (Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides)
- Publication en 1994 : *Design Patterns: Elements of Reusable Object-Oriented Software*

1-1-1-2

Contexte historique

- Christopher Alexander crée des “motifs” réutilisables en architecture
- Les 4 informaticiens adaptent cette idée pour la programmation orientée objet
- Formalisation de 23 patrons dans un ouvrage devenu référence
- Bilan : bases solides pour une conception logicielle modulaire et réutilisable

1-1-1-3

Pourquoi utiliser les Design Patterns du Gang of Four ?

- **Communiquer efficacement** avec un vocabulaire commun
- **Éviter la redondance** en réutilisant des solutions éprouvées
- **Améliorer la maintenabilité** grâce à un code plus clair et extensible

1-1-1-4

Les 3 catégories de patrons de conception

Catégorie	Description	Exemples courants
Créationnels	Gestion de la création d'objets	Singleton, Factory Method, Builder
Structurels	Composition des classes et objets	Adapter, Composite, Decorator
Comportementaux	Communication entre objets	Observer, Strategy, Command

1-1-1-5

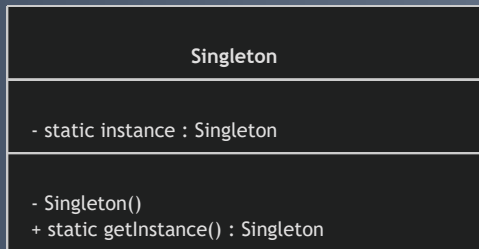
Exemple : Pattern Singleton

- Assure une seule instance d'une classe
- Point d'accès global à cette instance

```
public class Singleton {  
    private static Singleton instance;  
  
    private Singleton() { }  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

1-1-1-6

Diagramme : Singleton



1-1-1-7

Impact et Actualité

- 20 ans après, reste une référence majeure en conception orientée objet
- Base essentielle malgré l'émergence de paradigmes et langages modernes
- Intégration fréquente dans les frameworks actuels (Java Spring, .NET, etc.)
- Fondement de nombreuses architectures logicielles contemporaines

1-1-1-8

Sources utilisées

- DigitalOcean, *Gang of 4 Design Patterns Explained*
- SE Radio, *Gang of Four – 20 Years Later*
- Developpez.com, *Design Patterns du Gang of Four appliqués à Java*

Conclusion

La formalisation des design patterns par le Gang of Four a structuré la conception logicielle, fournissant des outils conceptuels indispensables qui perdurent encore aujourd'hui.

1-1-2-2

Genèse des design patterns en génie logiciel

Origines conceptuelles externes

- Inspirés de l'architecture avec Christopher Alexander dans les années 1970.
- "Patrons" = langage commun pour résoudre des problèmes de conception.
- Transposition au logiciel dans les années 80-90 face à la complexité croissante.

1-1-2-3

Contexte logiciel dans les années 1980

- Montée de la Programmation Orientée Objet (POO).
- Nouveaux défis : modularité, évolutivité, communication inter-objets.
- Absence de cadre formalisé pour garantir des designs efficaces.
- Design patterns = réponse pragmatique, modèles réutilisables pour problèmes typiques.

1-1-2-4

Formalisation par le Gang of Four (GoF)

- En 1994, sortie du livre *Design Patterns: Elements of Reusable Object-Oriented Software*.
- Auteurs : Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides.
- 23 patrons classés en :
 - Créationnels
 - Structurels
 - Comportementaux
- Langage commun et boîte à outils conceptuelle pour développeurs.

Exemple : Factory Method (pattern créationnel)

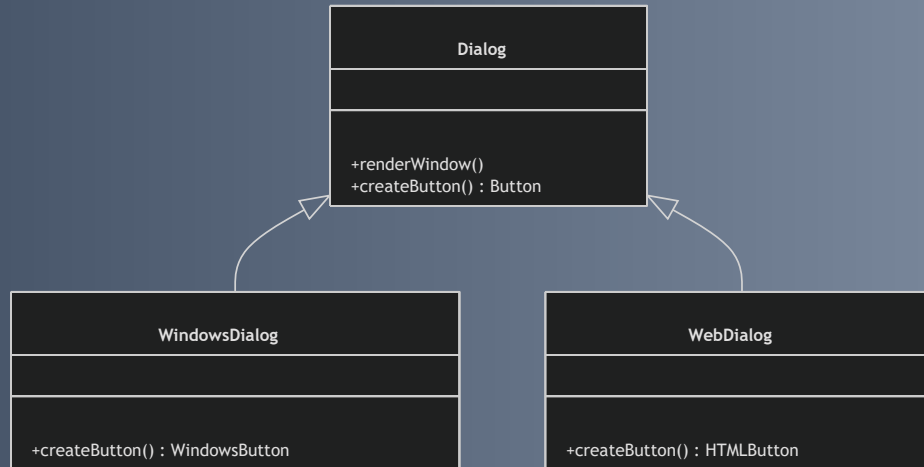
Problème posé : Créer des objets sans connaître leur classe concrète à l'avance.

Solution : Déléguer la création à une sous-classe via une méthode "fabrique".

```
abstract class Dialog {  
    public void renderWindow() {  
        Button okButton = createButton();  
        okButton.render();  
    }  
    public abstract Button createButton();  
}  
  
class WindowsDialog extends Dialog {  
    public Button createButton() {  
        return new WindowsButton();  
    }  
}  
  
class WebDialog extends Dialog {  
    public Button createButton() {  
        return new HTMLButton();  
    }  
}
```

1-1-2-6

Diagramme illustrant Factory Method



1-1-2-7

Bénéfices de l'émergence des design patterns

- **Réutilisabilité** : solutions éprouvées capitalisées.
- **Communicabilité** : vocabulaire unifié facilitant la collaboration.
- **Robustesse & maintenance** : architectures claires, évolutives, testables.
- **Standardisation** : facilite la montée en compétence par des modes de conception répandus.

1-1-2-8

Sources utilisées

- Les Design Patterns - Le LIP6
- Rheinwerk Computing, "What Are Design Patterns? History, Origins, and Software Development Impact"
- Génie Logiciel - GitBook, "Patrons logiciels"
- Wikipédia, "Patron de conception"

Conclusion

- Design patterns = nécessité face à la complexité croissante des systèmes orientés objet.
- Apport essentiel : cadre conceptuel pour une conception objet maintenable et évolutive.
- Incontournables pour toute démarche rigoureuse de génie logiciel moderne.

Slide 1-1-3-2

Réutilisabilité : capitaliser sur des solutions éprouvées

Problématique

- Réutiliser évite duplication et effort de conception
- Exige des structures génériques, flexibles et bien définies

Réutilisabilité: Apport des design patterns

- **Patterns** = solutions éprouvées à problèmes récurrents, modèles conceptuels adaptables
- **Modularité** : Strategy change dynamiquement le comportement sans modifier l'objet
- **Découplage** : Observer, Mediator facilitent communication entre objets indépendants
- **Généricité** : Factory Method centralise la création d'objets, simplifie l'extension du système
- Résultat : réduction du temps de développement, réutilisation efficace

Slide 1-1-3-4

Maintenance : faciliter les évolutions et corrections

Défi

- Logiciels en évolution constante, maintenance fréquente et complexe
- Design rigide, peu clair ou trop couplé complique les modifications

Maintenance : Contribution des design patterns

- Principe OCP : ouvert à l'extension, fermé à la modification
- Isolation des responsabilités : Decorator ajoute des fonctionnalités sans modifier le code source
- Réduction du couplage : interactions fragmentées, code plus clair et facile à modifier

Exemple pratique : Le pattern Observer

- Relation de dépendance 1-n : un **Sujet** notifie automatiquement ses **Observateurs** en cas de changement
- Sujet et observateurs découplés

```
interface Observer {  
    void update();  
}  
  
class Subject {  
    private List<Observer> observers = new ArrayList<>();  
  
    public void attach(Observer o) {  
        observers.add(o);  
    }  
  
    public void notifyObservers() {  
        for (Observer o : observers) {  
            o.update();  
        }  
    }  
}
```

Diagramme - Pattern Observer

Slide 1-1-3-9

Avantages du pattern Observer pour maintenance et réutilisation

- Sujet ne connaît pas les détails des observateurs (découplage)
- Ajout/suppression d'observateurs sans modifier le sujet
- Réutilisable dans divers contextes : interfaces graphiques, systèmes d'événements, etc.

1-1-3-10

Synthèse : Impact des Design Patterns

Aspect	Impact principal
Réutilisabilité	Solutions génériques, modularité, découplage
Maintenance	Extensibilité sans modification, responsabilités isolées, faible couplage

1-1-3-11

Sources utilisées

- Oracle, "Design Patterns and Principles" : <https://docs.oracle.com/javase/tutorial/java/concepts/designpatterns.html>
- Refactoring Guru, "Observer Design Pattern" : <https://refactoring.guru/design-patterns/observer>
- IBM Developer, "Why are design patterns important?" : <https://developer.ibm.com/articles/design-patterns-in-software-engineering/>
- Wikipedia, "Design pattern (computer science)" : [https://en.wikipedia.org/wiki/Design_pattern_\(computer_science\)](https://en.wikipedia.org/wiki/Design_pattern_(computer_science))

Conclusion

- Les design patterns sont un levier essentiel pour construire des logiciels flexibles et durables
- Ils maîtrisent la complexité tout en favorisant :
 - Réutilisation aisée du code
 - Maintenance facilitée et évolutions simplifiées

1-2-1-1

Patterns de création : gestion de l'instanciation des objets

Introduction

- La création d'objets est fondamentale en programmation orientée objet.
- Appeler directement les constructeurs peut rendre le code rigide et difficile à maintenir.
- **Patterns de création** : mécanismes abstraits et flexibles pour gérer l'instanciation.
- Cachent la complexité de la création.

1-2-1-2

Qu'est-ce qu'un pattern de création ?

- Patron de conception ciblant :
 - La reconnaissance des objets à créer,
 - Leur manière d'instanciation,
 - Les étapes de leur construction.
- Avantages :
 - Isolation de la logique de création,
 - Gestion d'objets complexes,
 - Flexibilité dans le choix de la classe concrète instanciée.

1-2-1-3

Principaux patterns de création : Singleton

- Garantit qu'une classe a une seule instance accessible globalement.

Exemple simplifié :

```
public class Singleton {  
    private static Singleton instance;  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

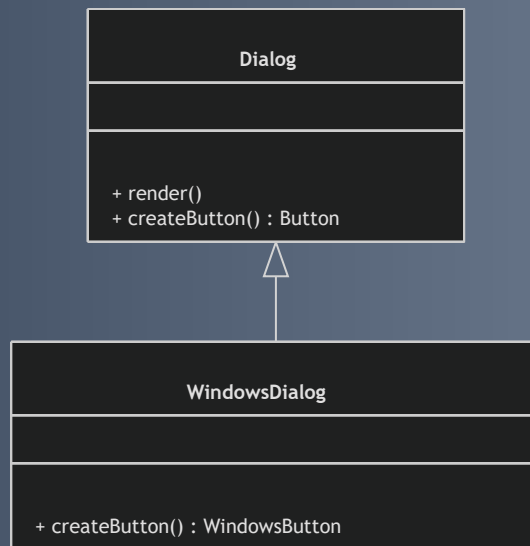
Principaux patterns de création : Factory Method

- Délègue la création d'objets à des sous-classes via une méthode abstraite.
- Permet de varier les types instanciés sans modifier le code client.

```
abstract class Dialog {  
    public void render() {  
        Button button = createButton();  
        button.render();  
    }  
    abstract Button createButton();  
}  
  
class WindowsDialog extends Dialog {  
    Button createButton() {  
        return new WindowsButton();  
    }  
}
```

1-2-1-6

Diagramme Factory Method



1-2-1-7

Principaux patterns de création : Abstract Factory

- Interface pour créer des familles d'objets liés sans spécifier leurs classes concrètes.

```
interface GUIFactory {  
    Button createButton();  
    Checkbox createCheckbox();  
}  
  
class WinFactory implements GUIFactory {  
    public Button createButton() { return new WinButton(); }  
    public Checkbox createCheckbox() { return new WinCheckbox(); }  
}
```

1-2-1-8

Principaux patterns de création : Builder

- Sépare la construction d'un objet complexe de sa représentation.
- Permet d'assembler l'objet pas à pas.

```
class CarBuilder {  
    private Car car = new Car();  
    public CarBuilder buildWheels(int count) { car.setWheels(count); return this; }  
    public CarBuilder buildEngine(Engine e) { car.setEngine(e); return this; }  
    public Car getResult() { return car; }  
}
```

1-2-1-9

Principaux patterns de création : Prototype

- Crée de nouveaux objets en clonant un objet prototype existant.
- Utile quand la création est coûteuse.

1-2-1-10

Synthèse des patterns créatifs

Pattern	Usage principal
Singleton	Instance unique globale
Factory Method	Délégation de création à sous-classes
Abstract Factory	Création de familles cohérentes d'objets
Builder	Construction d'objets complexes en étapes
Prototype	Création par clonage d'objets existants

1-2-1-11

Intérêt pour la conception logicielle

- **Flexibilité** : Modifier la création sans changer le code client.
- **Encapsulation** : Logique de création isolée des classes spécifiques.
- **Réduction des dépendances** : Favorise découplage et extensibilité.

1-2-1-12

Sources utilisées

- Refactoring Guru, "Creational Design Patterns":
<https://refactoring.guru/design-patterns/creational-patterns>
- Oracle, "Design Patterns - Creational":
<https://docs.oracle.com/javase/tutorial/java/concepts/designpatterns.html>
- Wikipedia, "Creational pattern":
https://en.wikipedia.org/wiki/Creational_pattern

Conclusion

- Les patrons de création organisent et maîtrisent le processus d'instanciation.
- Réduisent la complexité et augmentent la robustesse des applications.

1-2-2-1

Patterns de structure : composition des classes et objets

- Optimisent les relations entre composants
- Facilitent réutilisation, maintenance, extension
- Organisent classes et objets en structures flexibles

1-2-2-2

Qu'est-ce qu'un pattern de structure ?

- Solution d'organisation et composition des classes/objets
- Permet architectures cohérentes, performantes, évolutives
- Traite héritage, composition, gestion des relations

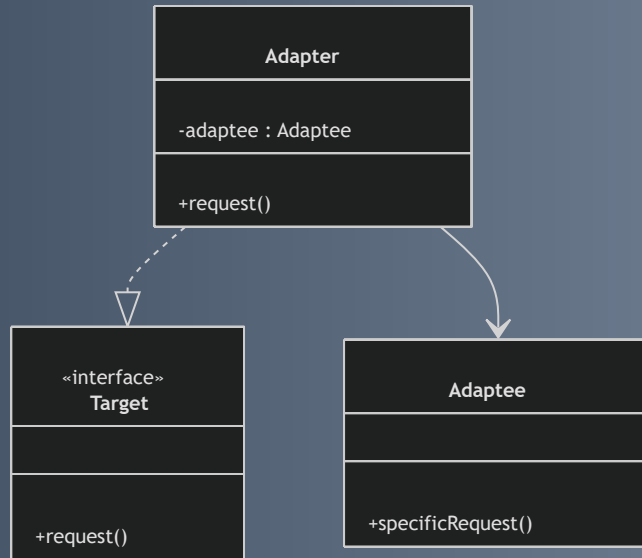
Pattern Adapter (Wrapper)

- Permet à des classes incompatibles de collaborer
- Convertit interface d'une classe en interface attendue

```
interface Target {  
    void request();  
}  
  
class Adaptee {  
    void specificRequest() {  
        System.out.println("Adaptee specific request");  
    }  
}  
  
class Adapter implements Target {  
    private Adaptee adaptee;  
  
    public Adapter(Adaptee a) {  
        this.adaptee = a;  
    }  
  
    public void request() {  
        adaptee.specificRequest(); // Traduction de l'appel
```

1-2-2-4

Diagramme Adapter

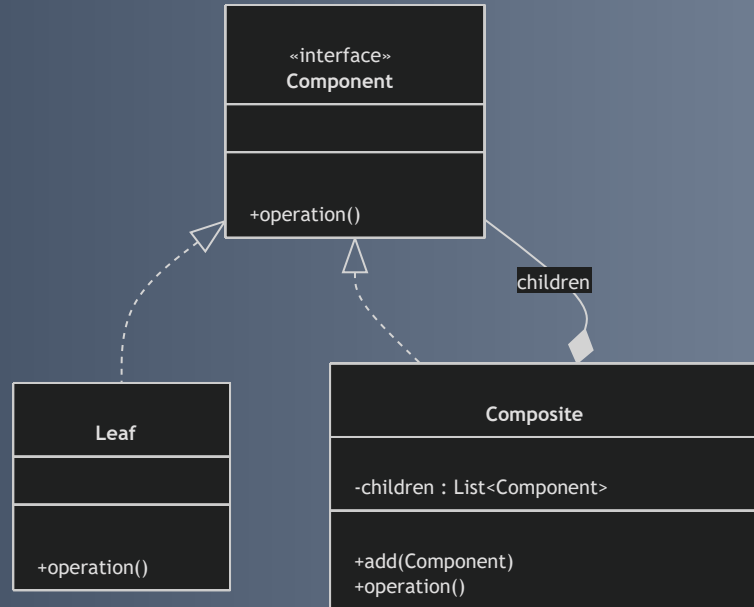


Pattern Composite

- Compose objets en structure arborescente (hiérarchie partie-tout)
- Client traite uniformément objets composites et feuilles

```
interface Component {  
    void operation();  
}  
  
class Leaf implements Component {  
    public void operation() {  
        System.out.println("Leaf operation");  
    }  
}  
  
class Composite implements Component {  
    private List<Component> children = new ArrayList<>();  
  
    public void add(Component c) { children.add(c); }  
    public void operation() {  
        for (Component c : children) {  
            c.operation();  
        }  
    }  
}
```


Diagramme Composite



1-2-2-7

Pattern Decorator

- Ajoute dynamiquement des responsabilités à un objet
- S'appuie sur composition et délégation

```
interface Coffee {  
    double cost();  
}  
  
class SimpleCoffee implements Coffee {  
    public double cost() { return 2; }  
}  
  
class MilkDecorator implements Coffee {  
    private Coffee coffee;  
  
    public MilkDecorator(Coffee c) { coffee = c; }  
  
    public double cost() { return coffee.cost() + 0.5; }  
}
```

1-2-2-8

Pattern Facade

- Propose une interface simplifiée pour un sous-système complexe
- Facilite l'usage et masque la complexité technique

1-2-2-9

Récapitulatif des patterns de structure

Pattern	Description	Cas d'usage typique
Adapter	Faire collaborer des interfaces incompatibles	Plugin tiers, intégration legacy
Composite	Travailler uniformément sur feuilles et groupes	Gestion hiérarchique (arbres UI)
Decorator	Ajouter/modifier dynamiquement fonctionnalités	Ajout d'attributs ou comportements
Facade	Simplifier interface d'un sous-système	Masquer complexité technique

1-2-2-10

Intérêt pour la conception logicielle

- Simplifie gestion des relations entre objets
- Favorise architectures modulaires, évolutives, testables
- Encourage composition plutôt que héritage pour plus de flexibilité

1-2-2-11

Sources

- Refactoring Guru, "Structural Design Patterns", <https://refactoring.guru/design-patterns/structural-patterns>
- Oracle, "Structural Patterns", <https://docs.oracle.com/javase/tutorial/java/concepts/designpatterns.html>
- Wikipedia, "Structural pattern", https://en.wikipedia.org/wiki/Structural_pattern

Conclusion

Les patterns de structure sont essentiels pour organiser la collaboration entre classes et objets dans un système complexe, garantissant souplesse et évolutivité.

1-2-3-1

Patterns de comportement : communication entre objets

- Définissent comment les objets interagissent et communiquent
- Organisent ces échanges pour :
 - améliorer flexibilité
 - favoriser réutilisabilité
 - faciliter évolution des applications orientées objet

1-2-3-2

Qu'est-ce qu'un pattern de comportement ?

- Spécifie la coopération entre objets :
 - distribution des responsabilités
 - échange de messages
- Préserve l'indépendance des objets
- Modulent la communication pour :
 - réduire couplage
 - simplifier coordination
 - favoriser architectures dynamiques

1-2-3-3

Pattern Observer

- Sujet notifie automatiquement plusieurs observateurs à chaque changement d'état
- Déconnexion entre sujet et observateurs, favorisant la modularité

```
interface Observer { void update(); }

class Subject {
    private List<Observer> observers = new ArrayList<>();
    public void attach(Observer o) { observers.add(o); }
    public void notifyObservers() {
        for (Observer o : observers) o.update();
    }
}
```

Diagramme Observer

1-2-3-5

Pattern Strategy

- Définit une famille d'algorithmes interchangeable dans des classes distinctes
- Permet à un objet de changer son comportement à l'exécution

```
interface Strategy { void execute(); }
class ConcreteStrategyA implements Strategy {
    public void execute() { System.out.println("Algorithme A"); }
}
class Context {
    private Strategy strategy;
    public void setStrategy(Strategy s) { strategy = s; }
    public void executeStrategy() { strategy.execute(); }
}
```

1-2-3-6

Autres patterns de comportement clés

- **Command** : encapsule une requête en objet, permet paramétrage, stockage, annulation d'actions
- **Iterator** : parcours séquentiel des éléments d'une collection sans exposer sa structure interne
- **Mediator** : centralise les interactions pour réduire dépendances directes et gérer communications complexes

1-2-3-7

Bénéfices des patterns de comportement

- **Réduction du couplage** : communication via interfaces, pas de dépendance forte
- **Extensibilité** : adaptation et extension des comportements facilitées
- **Clarté** : responsabilités bien réparties, compréhension améliorée
- **Modularité** : tâches complexes décomposées en interactions simples

1-2-3-8

Synthèse rapide

Pattern	Description	Exemple d'usage
Observer	Notification d'un changement à plusieurs	Système d'événements UI
Strategy	Algorithmes interchangeables	Choix dynamique d'algorithme
Command	Encapsulation d'une action	File d'attente de commandes
Iterator	Parcours d'une collection	Parcours uniforme de collections
Mediator	Centralisation des communications	Gestion de workflow complexe

1-2-3-9

Sources utilisées

- Refactoring Guru, "Behavioral Design Patterns", <https://refactoring.guru/design-patterns/behavioral-patterns>
- Oracle, "Behavioral Patterns", <https://docs.oracle.com/javase/tutorial/java/concepts/designpatterns.html>
- Wikipedia, "Behavioral pattern", https://en.wikipedia.org/wiki/Behavioral_pattern

Conclusion

- Les patterns de comportement modèlent les collaborations entre objets
- Standardisent les interactions pour construire des logiciels :
 - modulaires
 - maintenables
 - évolutifs

1-3-1-1

Principes SOLID : Fondamentaux de la conception objet

- Ensemble de 5 principes pour concevoir des logiciels faciles à maintenir et extensibles
- Structuration claire du code pour :
 - Faciliter la compréhension
 - Favoriser la réutilisation
 - Permettre l'évolution agile

1-3-1-2

1. Single Responsibility Principle (SRP)

Principe de responsabilité unique

- Une classe = une seule responsabilité, une seule raison de changer
- Séparation des fonctionnalités dans des classes distinctes

```
class InvoicePrinter {  
    public void printInvoice(Invoice invoice) { /* Impression */ }  
}  
class InvoicePersistence {  
    public void saveToDatabase(Invoice invoice) { /* Sauvegarde */ }  
}
```

2. Open/Closed Principle (OCP)

Ouvert à l'extension, fermé à la modification

- Ajouter des comportements sans modifier le code existant
- Utilisation d'abstractions (interfaces, classes abstraites)

Exemple stratégie :

```
interface Shape {  
    double area();  
}  
  
class Circle implements Shape {  
    double radius;  
    public double area() { return Math.PI * radius * radius; }  
}  
  
class Rectangle implements Shape {  
    double width, height;  
    public double area() { return width * height; }  
}
```

3. Liskov Substitution Principle (LSP)

Substituabilité de Liskov

- Les classes dérivées peuvent remplacer les classes mères sans changer le comportement
- Violation typique : héritage incompatible

Exemple problématique :

```
class Rectangle {  
    void setWidth(int w);  
    void setHeight(int h);  
}  
class Square extends Rectangle {  
    void setWidth(int w) {  
        // modifie largeur et hauteur  
    }  
    void setHeight(int h) {  
        // modifie largeur et hauteur  
    }  
}
```

Square ne respecte pas les attentes d'un Rectangle.

4. Interface Segregation Principle (ISP)

Séparation des interfaces

- Préférer plusieurs interfaces spécifiques à une interface unique générale
- Les clients dépendent uniquement des méthodes nécessaires

```
interface Printer {  
    void print();  
}  
  
interface Scanner {  
    void scan();  
}  
  
class MultiFunctionPrinter implements Printer, Scanner {  
    public void print() {}  
    public void scan() {}  
}  
  
class OldPrinter implements Printer {  
    public void print() {}  
}
```

5. Dependency Inversion Principle (DIP)

Inversion des dépendances

- Modules haut niveau ne dépendent pas des modules bas niveau
- Tous dépendent d'abstractions (interfaces)

```
interface MessageService {  
    void sendMessage(String message);  
}  
  
class EmailService implements MessageService {  
    public void sendMessage(String message) { /* envoyer email */ }  
}  
  
class Notification {  
    private MessageService service;  
    public Notification(MessageService s) { service = s; }  
    public void notifyUser(String msg) {  
        service.sendMessage(msg);  
    }  
}
```

1-3-1-8

Résumé des principes et objectifs

Principe	But principal
SRP	Une classe = une responsabilité
OCP	Étendre sans modifier le code
LSP	Substituabilité garantie
ISP	Interfaces fines et spécifiques
DIP	Dépendre d'abstractions, pas de classes concrètes

Sources

- Robert C. Martin, "Principles of Object-Oriented Design"
<https://blog.cleancoder.com/uncle-bob/2014/05/08/SingleResponsibilityPrinciple.html>
- Refactoring Guru, "SOLID Principles"
<https://refactoring.guru/design-patterns/solid>
- Wikipedia, "SOLID"
<https://en.wikipedia.org/wiki/SOLID>

Conclusion

Les principes SOLID garantissent :

- Une architecture claire et modulaire
- Une maintenance facilitée
- Une évolution du logiciel agile et sécurisée

Grâce à une séparation nette des responsabilités et un découplage efficace entre composants.

1-4-1-1

Article 1-4-1 : Définition et exemples d'anti-patterns

Introduction

- Un **anti-pattern** est une solution récurrente à un problème de conception/développement qui semble pertinente mais provoque :
 - Complexité accrue
 - Maintenance difficile
 - Dette technique élevée
- Comprendre les anti-patterns aide à éviter des pièges courants en conception logicielle.

1-4-1-2

Qu'est-ce qu'un anti-pattern ?

- Contrairement aux **design patterns** : solutions efficaces et éprouvées
- Un **anti-pattern** est une solution trompeuse ou incorrecte à un problème
- Peut se manifester en :
 - Conception du code
 - Architecture
 - Gestion de projet
- Cause plus de mal que de bien.

1-4-1-3

Exemples courants d'anti-patterns en conception logicielle

1. God Object (Objet Dieu)

- Une classe concentrant trop de responsabilités
- Violation du principe de responsabilité unique (SRP)
- Impact : difficile à modifier, tester, réutiliser

```
class GodObject {  
    public void manageUI() { /* Code UI */ }  
    public void processBusinessLogic() { /* Logique métier */ }  
    public void accessDatabase() { /* Accès base de données */ }  
}
```

1-4-1-4

Exemples d'anti-patterns (suite)

2. Spaghetti Code

- Code désordonné, sans structure claire
- Logique entremêlée
- Compréhension et maintenance quasi impossibles

3. Golden Hammer

- Utilisation systématique d'une solution connue
- Appliquer toujours le même design pattern, quel que soit le contexte

1-4-1-5

Exemples d'anti-patterns (fin)

4. Copy-Paste Programming

- Dupliquer le code au lieu de refactoriser ou extraire une fonctionnalité
- Maintenance lourde et sujette aux erreurs

5. Lava Flow

- Code ou fonctionnalités obsolètes ou inutilisées
- Par peur de casser quelque chose

1-4-1-7

Comment éviter les anti-patterns ?

- Respecter les principes de conception, notamment **SOLID**
- Favoriser la modularité et la séparation des responsabilités
- Refactoriser régulièrement le code
- Documenter clairement les choix techniques
- S'appuyer sur les design patterns pertinents

1-4-1-8

Sources utilisées

- Refactoring Guru, "Anti-patterns", <https://refactoring.guru/anti-patterns>
- Wikipedia, "Anti-pattern", <https://en.wikipedia.org/wiki/Anti-pattern>
- Martin Fowler, "Refactoring: Improving the Design of Existing Code", <https://martinfowler.com/books/refactoring.html>

Conclusion

- Reconnaître les anti-patterns est clé pour la qualité du code
- Identification précoce permet de corriger avant accumulation de dette technique
- Favorise des logiciels durables et évolutifs

1-4-2-1

Reconnaître les mauvaises pratiques courantes

- Les anti-patterns nuisent à la qualité, maintenabilité et robustesse.
- Les identifier permet correction rapide et adoption de bonnes pratiques.

Mauvaises pratiques courantes

1. Code Duplication (Duplication de code)

- Copier-coller de blocs identiques ou similaires.
- **Conséquences :**
 - Maintenance laborieuse (bug corrigé partiellement).
 - Volume de code inutilement augmenté.

2. God Object (Classe “Dieu”)

- Une classe concentre la majorité des responsabilités.
- **Problème :** code rigide, difficile à comprendre et modifier.

3. Shotgun Surgery (Chirurgie de fusil à pompe)

- Modifier une fonctionnalité nécessite changer de nombreuses petites parties dispersées.
- **Conséquence :** risque élevé d'erreurs lors des modifications.

1-4-2-5

4. Magic Numbers (Nombres magiques)

- Usage de constantes littérales sans explication dans le code.

Exemple :

```
if (age > 18) { // 18 est un nombre magique
    // ...
}
```

- **Bonne pratique :** définir une constante explicite

```
final int ADULT_AGE = 18;
if (age > ADULT_AGE) {
    // ...
}
```

1-4-2-6

5. Feature Envy (Envie de fonctionnalité)

- Un objet accède trop aux données d'un autre, brisant l'encapsulation.

6. Lava Flow

- Code mort ou incompris laissé dans le projet par crainte de le retirer.

7. Reinventing the Wheel

- Réimplémenter des fonctionnalités standards au lieu d'utiliser des bibliothèques éprouvées.

1-4-2-9

Comment reconnaître ces mauvaises pratiques ?

- **Symptômes** : classes lourdes, duplications, dépendances fortes.
- **Difficultés** : corrections répétées, bugs fréquents après modification.
- **Outils d'analyse statique** : SonarQube, ESLint, etc.
- **Revue de code rigoureuse** : détection humaine essentielle.

1-4-2-11

Bonnes pratiques pour limiter ces anti-patterns

- Appliquer le **principe de responsabilité unique (SRP)**.
- Refactorer régulièrement pour éliminer duplications.
- Utiliser des constantes symboliques plutôt que nombres magiques.
- Favoriser l'encapsulation, limiter accès direct aux données.
- S'appuyer sur des bibliothèques standardisées.
- Effectuer des revues de code systématiques.

1-4-2-12

Sources utilisées

- Refactoring Guru, "Code Smells and Anti-patterns"
- Martin Fowler, "Refactoring: Improving the Design of Existing Code"
- Wikipedia, "Code Smell"
- SonarSource, "SonarQube documentation"

Conclusion

Reconnaître et éliminer les mauvaises pratiques améliore :

- La maintenance
- L'évolution du projet
- La collaboration entre développeurs

Adopter une architecture saine facilite le succès des projets logiciels.

2-1-1-2

Description du pattern Singleton

- Contrôle strict de l'instanciation pour 1 seul objet
- Évite la duplication d'instances (ex : connexions base de données, gestionnaires de configuration)

Caractéristiques clés :

- Instance unique accessible globalement
- Contrôle strict de la création d'objets
- Usage fréquent pour ressources partagées (connexion BD, cache, logs)

Implémentation classique

```
public class Singleton {  
    // Instance unique et statique  
    private static Singleton instance;  
  
    // Constructeur privé empêche l'instanciation externe  
    private Singleton() {}  
  
    // Méthode accès à l'unique instance  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Variante thread-safe : double-checked locking

- Problème : en multithread, plusieurs instances peuvent être créées
- Solution : synchronisation fine avec double vérification

```
public class Singleton {  
    private static volatile Singleton instance;  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            synchronized(Singleton.class) {  
                if (instance == null) {  
                    instance = new Singleton();  
                }  
            }  
        }  
        return instance;  
    }  
}
```


2-1-1-6

Utilisations classiques du Singleton

- Gestionnaire de configuration
- Logger centralisé
- Pool de connexion
- Cache global

Limites importantes :

- Couplage fort entre classes utilisant le singleton
- Difficultés de test (mock, gestion état global)
- Risques en multithread mal gérés

2-1-1-7

Alternatives au Singleton

- Injection de dépendances avec scope singleton
- Objets immuables partagés
- Conteneurs de gestion des ressources (ex : Spring)

2-1-1-8

Sources Utilisées

- Refactoring Guru, "Singleton Design Pattern"
- Oracle Java Tutorials, "Singleton Pattern"
- Wikipedia, "Singleton pattern"

Conclusion

- Singleton contrôle la création d'une instance unique
- Permet un accès global à cette instance
- Usage à considérer avec soin pour éviter couplage fort, problèmes de test et concurrence
- Alternatives existent pour une meilleure gestion des dépendances et ressources partagées

Cas d'usage typiques du Singleton

1. Gestionnaire de configuration

- Accès centralisé et uniforme aux paramètres de configuration
- Évite la création de multiples objets pour les paramètres

```
public class ConfigurationManager {  
    private static ConfigurationManager instance;  
    private Properties config;  
  
    private ConfigurationManager() {  
        config = new Properties();  
        // Chargement des paramètres depuis un fichier  
    }  
  
    public static ConfigurationManager getInstance() {  
        if (instance == null) {  
            instance = new ConfigurationManager();  
        }  
        return instance;  
    }  
  
    public String getProperty(String key) {  
        return config.getProperty(key);  
    }  
}
```

2-1-2-3

2. Logger centralisé

- Tous les logs transitent par une instance unique cohérente
- Facilite la gestion centralisée des enregistrements

```
public class Logger {  
    private static Logger instance;  
  
    private Logger() {  
        // initialisation, ouverture fichier etc.  
    }  
  
    public static Logger getInstance() {  
        if (instance == null) instance = new Logger();  
        return instance;  
    }  
  
    public void log(String message) {  
        System.out.println(message);  
    }  
}
```

3. Connexion à une base de données (exemple basique)

- Maintenir une unique connexion ou un pool géré
- Évite surcharge et conflits liés à plusieurs connexions simultanées

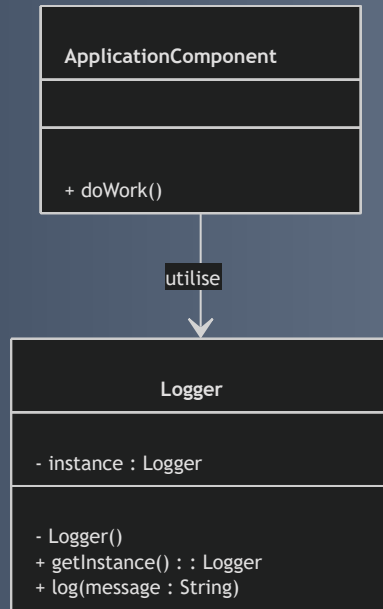
```
public class DatabaseConnection {  
  
    private static DatabaseConnection instance;  
  
    private final String URL = "jdbc:mysql://localhost:3306/ma_base";  
    private final String USER = "root";  
    private final String PASSWORD = "password";  
  
    private DatabaseConnection() {  
        try {  
            connection = DriverManager.getConnection(URL, USER, PASSWORD);  
        } catch (SQLException e) {  
            throw new RuntimeException("Erreur de connexion à la base de données", e);  
        }  
    }  
  
    public static DatabaseConnection getInstance() {  
        if (instance == null) {  
            instance = new DatabaseConnection();  
        }  
        return instance;  
    }  
}
```

2-1-2-5

4. Cache global

- Cache partagé accessible par plusieurs composants
- Permet d'éviter des recalculs coûteux
- Stocke des données fréquemment utilisées

Diagramme illustrant le cas d'un Logger



2-1-2-7

Précautions à prendre

- **Environnements multi-thread**

Utiliser une implémentation thread-safe pour éviter plusieurs instances

- **Testabilité**

Difficulté à mocker ; préférer l'injection de dépendances

- **Couplage global**

Minimiser les dépendances directes pour préserver modularité et flexibilité

Sources utilisées

- Refactoring Guru, "Singleton Design Pattern"
<https://refactoring.guru/design-patterns/singleton>
- Baeldung, "Understanding the Singleton Pattern in Java"
<https://www.baeldung.com/java-singleton>
- Oracle Tutorials, "Singleton Pattern"
<https://docs.oracle.com/javase/tutorial/java/javaOO/singleton.html>

Conclusion

- Le pattern Singleton impose une instance unique partagée
- Usage raisonné : assure une coordination fiable
- Simplifie la gestion de ressources communes dans les applications

2-1-3-1

Limites et alternatives au Singleton

Introduction

- Le pattern **Singleton** est parfois considéré comme un anti-pattern
- Ses limites se manifestent surtout dans des architectures complexes ou multi-threadées
- Identifier ces faiblesses et explorer des alternatives aide à concevoir des logiciels plus modulaires, testables, évolutifs

2-1-3-2

Limites du pattern Singleton

1. Couplage fort et global

- Instance unique globale accessible partout
- Crée un couplage fort
- Rend difficile le remplacement ou la mise en place de mocks lors des tests unitaires

2. Difficultés de testabilité

- Singleton conserve un état global partagé
- Introduit des effets de bord entre tests
- Complexifie l'utilisation de mocks ou stubs

2-1-3-3

3. Problèmes en environnement multi-thread

- Implémentation incorrecte peut créer plusieurs instances simultanément
- Compromet la garantie d'unicité du Singleton

4. Gestion de la durée de vie et des dépendances

- Singleton contrôle lui-même sa création et destruction
- Limite le contrôle global sur la durée de vie des objets
- Inadapté dans des architectures complexes

2-1-3-6

Alternatives modernes au Singleton

1. Injection de dépendances (DI)

- Externalise la création des objets vers un conteneur ou la couche d'initialisation
- **Avantages :**
 - Contrôle précis de l'instanciation et de la durée de vie
 - Facilite remplacement et tests avec implémentations alternatives

```
class Service {  
    private final Config config;  
    public Service(Config config) {  
        this.config = config;  
    }  
}
```

- Ici `Config` est injecté, favorisant modularité et testabilité

2-1-3-7

2. Utilisation de classes utilitaires statiques

- Pour certains besoins (ex : gestion de logs)
- Méthodes statiques simples suffisent
- Évite la complexité d'un Singleton

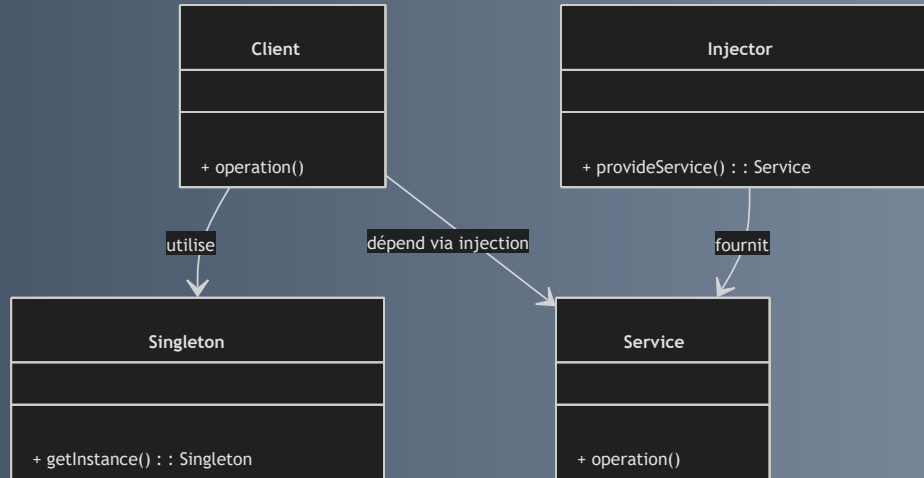
3. Objets immutables partagés

- Objets créés immuables au démarrage de l'application
- Passés en référence
- Assurent l'absence d'effets de bord

4. Conteneurs de services (Service Locator)

- Conteneur central fournissant des instances singleton ou à durée contrôlée
- Délègue le cycle de vie des objets
- N'expose pas d'accès global direct

Diagramme : comparaison Singleton vs Injection de dépendances



2-1-3-11

Synthèse

Aspect	Singleton	Injection de dépendances
Contrôle d'instance	Automatique, unique	Externe, configurable
Testabilité	Difficile (état global)	Facile (mock facile)
Couplage	Fort, global	Faible, explicite
Concurrence	Risques si mal implémenté	Géré par conteneur

2-1-3-12

Sources utilisées

- Martin Fowler, "Inversion of Control Containers and the Dependency Injection pattern", <https://martinfowler.com/articles/injection.html>
- Refactoring Guru, "Singleton Anti-pattern", <https://refactoring.guru/design-patterns/singleton#anti-pattern>
- Baeldung, "Singleton Design Pattern and Its Alternatives", <https://www.baeldung.com/java-singleton>
- Wikipedia, "Singleton pattern", https://en.wikipedia.org/wiki/Singleton_pattern

Conclusion

- Le Singleton reste un pattern utile mais à utiliser avec prudence
- Pour les projets complexes, privilégier des alternatives comme l'injection de dépendances
- Cela améliore modularité, testabilité et maintenance des applications

2-2-1-1

Principe de Factory Method

Introduction

- Pattern de création déléguant l'instanciation aux sous-classes
- Définit une interface pour créer un objet
- Classes dérivées choisissent la classe concrète à instancier
- Favorise flexibilité et extensibilité

2-2-1-2

Principe du Factory Method

- Résout le couplage client-classes concrètes
- Le client appelle une méthode de création (factory method)
- Les sous-classes redéfinissent cette méthode pour instancier un produit concret

Structure basique

- **Produit (Product)** : interface ou classe abstraite
- **ConcreteProduct** : implémentation concrète
- **Créateur (Creator)** : classe abstraite avec factoryMethod()
- **ConcreteCreator** : dérivée de Creator, implémente factoryMethod()

Exemple : Interface et Produits Concrets

```
// Produit
interface Transport {
    void deliver();
}

// Produits concrets
class Truck implements Transport {
    public void deliver() {
        System.out.println("Delivery by land in a truck");
    }
}

class Ship implements Transport {
    public void deliver() {
        System.out.println("Delivery by sea in a ship");
    }
}
```

Exemple : Créateurs

```
// Créateur abstrait
abstract class Logistics {
    // Méthode factory
    abstract Transport createTransport();

    public void planDelivery() {
        Transport transport = createTransport();
        transport.deliver();
    }
}

// Créateurs concrets
class RoadLogistics extends Logistics {
    Transport createTransport() {
        return new Truck();
    }
}

class SeaLogistics extends Logistics {
    Transport createTransport() {
        return new Ship();
    }
}
```

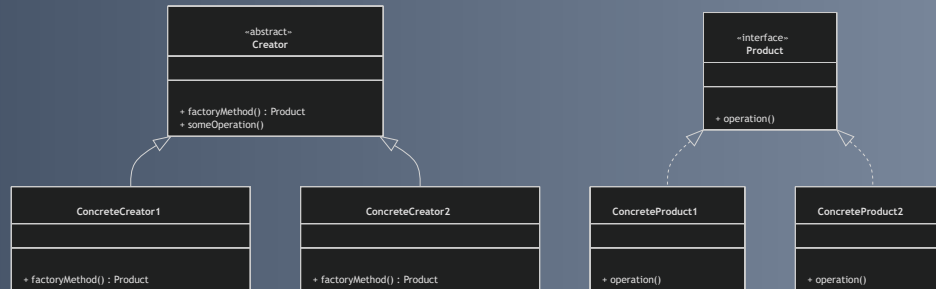
2-2-1-5

Exemple : Utilisation

```
public class Main {  
    public static void main(String[] args) {  
        Logistics logistics = new RoadLogistics();  
        logistics.planDelivery();  
  
        logistics = new SeaLogistics();  
        logistics.planDelivery();  
    }  
}
```

2-2-1-6

Diagramme : Factory Method



2-2-1-7

Avantages du Factory Method

- Découple le code client des classes concrètes
- Facilite l'ajout de nouveaux types sans modifier le client
- Respecte le principe OCP (Open/Closed Principle)

2-2-1-8

Contextes typiques d'utilisation

- Quand la classe ne peut anticiper le type concret nécessaire
- Pour isoler la création dans une hiérarchie de classes
- Dans des frameworks ou bibliothèques avec points d'extension par héritage

2-2-1-9

Sources utilisées

- Refactoring Guru, "Factory Method", <https://refactoring.guru/design-patterns/factory-method>
- Wikipedia, "Factory method pattern", https://en.wikipedia.org/wiki/Factory_method_pattern
- Gamma et al., "Design Patterns: Elements of Reusable Object-Oriented Software", Addison-Wesley, 1994

Conclusion

Le pattern Factory Method :

- Injecte de la souplesse dans la conception orientée objet
- Isole la création des objets dans une hiérarchie
- Facilite l'adaptabilité et la maintenance du code

2-2-2-1

Abstract Factory : principe général

- Pattern de création fournissant une **interface pour créer des familles d'objets liés**.
- Différent du Factory Method :
 - Factory Method crée un seul produit.
 - Abstract Factory crée plusieurs produits connexes devant fonctionner ensemble.

2-2-2-2

Composants clés de l'Abstract Factory

- **AbstractFactory** : interface pour créer chaque type de produit.
- **ConcreteFactory** : crée les objets concrets d'une famille spécifique.
- **AbstractProduct** : interface ou classe abstraite pour chaque type de produit.
- **ConcreteProduct** : implémentations concrètes des produits.
- **Client** : utilise l'AbstractFactory et les AbstractProducts sans connaître les classes concrètes.

2-2-2-3

Exemple : GUI multi-plateforme abstraite

- Produits abstraits : `Button`, `Checkbox` (interfaces).
- Produits concrets Windows : `WindowsButton`, `WindowsCheckbox`.
- Produits concrets MacOS : `MacOSButton`, `MacOSCheckbox`.
- Interface usine : `GUIFactory` crée `Button` et `Checkbox`.
- Usines concrètes : `WindowsFactory`, `MacOSFactory`.
- Client (`Application`) créé avec une `GUIFactory` pour instancier les bons produits selon l'OS.

2-2-2-4

Interface des produits

```
interface Button {  
    void paint();  
}  
  
interface Checkbox {  
    void paint();  
}
```

2-2-2-5

Produits concrets Windows

```
class WindowsButton implements Button {  
    public void paint() {  
        System.out.println("Render a button in Windows style.");  
    }  
}  
  
class WindowsCheckbox implements Checkbox {  
    public void paint() {  
        System.out.println("Render a checkbox in Windows style.");  
    }  
}
```


2-2-2-6

Produits concrets MacOS

```
class MacOSButton implements Button {  
    public void paint() {  
        System.out.println("Render a button in MacOS style.");  
    }  
}  
  
class MacOSCheckbox implements Checkbox {  
    public void paint() {  
        System.out.println("Render a checkbox in MacOS style.");  
    }  
}
```

Abstract Factory & Concrete Factories

```
interface GUIFactory {  
    Button createButton();  
    Checkbox createCheckbox();  
}  
  
class WindowsFactory implements GUIFactory {  
    public Button createButton() {  
        return new WindowsButton();  
    }  
    public Checkbox createCheckbox() {  
        return new WindowsCheckbox();  
    }  
}  
  
class MacOSFactory implements GUIFactory {  
    public Button createButton() {  
        return new MacOSButton();  
    }  
    public Checkbox createCheckbox() {  
        return new MacOSCheckbox();  
    }  
}
```

Client et utilisation

```
class Application {
    private Button button;
    private Checkbox checkbox;

    public Application(GUIFactory factory) {
        button = factory.createButton();
        checkbox = factory.createCheckbox();
    }

    public void paint() {
        button.paint();
        checkbox.paint();
    }
}

---

public class Demo {
    public static void main(String[] args) {
        GUIFactory factory;
        String osName = System.getProperty("os.name").toLowerCase();

        if (osName.contains("mac")) {
            factory = new MacOSFactory();
        } else {
            factory = new WindowsFactory();
        }
    }
}
```

2-2-2-9



2-2-2-10

Avantages du pattern Abstract Factory

- Garantit la **cohérence des produits** dans une même famille.
- Encapsule la **création d'objets complexes**.
- Facilite l'**extension** avec de nouvelles familles sans modifier le client.
- Favorise l'**indépendance des clients par rapport aux classes concrètes**.

2-2-2-11

Cas d'utilisation classiques

- Interfaces graphiques multi-plateformes (widgets adaptés).
- Systèmes requérant la création d'**objets liés et cohérents** (UI, bases de données, services).
- Quand une famille d'objets doit être utilisée et maintenue **ensemble**.

2-2-2-12

Sources utilisées

- Refactoring Guru, "Abstract Factory", <https://refactoring.guru/design-patterns/abstract-factory>
- Wikipedia, "Abstract factory pattern", https://en.wikipedia.org/wiki/Abstract_factory_pattern
- Gamma et al., "Design Patterns: Elements of Reusable Object-Oriented Software", Addison-Wesley, 1994

Conclusion

Le **pattern Abstract Factory** gère la création d'ensembles cohérents d'objets

sans exposer les classes concrètes, ce qui améliore la **modularité, flexibilité et maintenabilité** du code.

2-2-3-1

Découplage de l'instanciation

Factory Method et Abstract Factory

- Objectif : séparer la création des objets des clients qui les utilisent
- Améliore la flexibilité, la maintenabilité et l'évolutivité du code
- Modèles de création (Design Patterns) éprouvés :
 - Factory Method
 - Abstract Factory

2-2-3-2

Pourquoi découpler l'instanciation ?

- Instancier des classes concrètes crée une dépendance forte
- Toute modification impacte le client → moins évolutif et moins maintenable
- Principe : isoler la création dans des classes/méthodes « factory »
- Le client travaille uniquement avec interfaces ou abstractions

2-2-3-3

Factory Method

- Délègue la création à une méthode définie dans une superclasse abstraite
- Sous-classes redéfinissent cette méthode pour instancier des objets concrets
- Le client utilise l'objet via l'interface sans connaître la classe concrète

Exemple simplifié – Factory Method

```
abstract class Dialog {  
    abstract Button createButton();  
  
    void render() {  
        Button okButton = createButton();  
        okButton.onClick();  
        okButton.render();  
    }  
}  
  
class WindowsDialog extends Dialog {  
    Button createButton() {  
        return new WindowsButton();  
    }  
}  
  
class HtmlDialog extends Dialog {  
    Button createButton() {  
        return new HtmlButton();  
    }  
}
```

2-2-3-5

Abstract Factory

- Fournit une interface pour créer plusieurs objets liés (famille de produits)
- Garantit la compatibilité entre objets créés ensemble
- Permet d'échanger toute une famille de produits sans modifier le client

Example simplify – Abstract Factory

```
interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}

class MacOSFactory implements GUIFactory {
    public Button createButton() { return new MacOSButton(); }
    public Checkbox createCheckbox() { return new MacOSCheckbox(); }
}

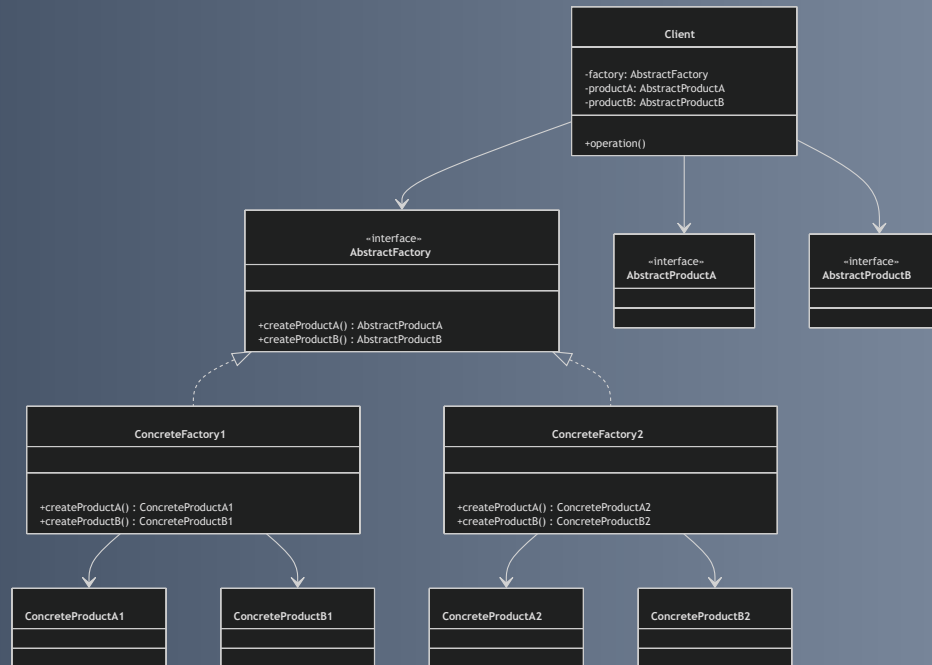
class WindowsFactory implements GUIFactory {
    public Button createButton() { return new WindowsButton(); }
    public Checkbox createCheckbox() { return new WindowsCheckbox(); }
}

class Application {
    private Button button;
    private Checkbox checkbox;

    public Application(GUIFactory factory) {
        this.button = factory.createButton();
        this.checkbox = factory.createCheckbox();
    }

    public void renderUI() {
        button.render();
        checkbox.render();
    }
}
```

Diagramme résumé du découplage



2-2-3-8

Bénéfices du découplage via Factory Method & Abstract Factory

- **Modularité accrue** : Client indépendant des classes concrètes
- **Facilité d'extension** : Ajouter un produit sans modifier le client
- **Respect des principes SOLID** : Ouvert/Fermé, responsabilité unique
- **Interopérabilité des familles de produits** : Objets compatibles garantis

Sources

- Refactoring Guru, “Factory Method”
<https://refactoring.guru/design-patterns/factory-method>
- Refactoring Guru, “Abstract Factory”
<https://refactoring.guru/design-patterns/abstract-factory>
- Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison Wesley, 1994
- Oracle Java Tutorials
<https://docs.oracle.com/javase/tutorial/java/landl/patterns.html>

Conclusion

- Découpler la création des objets est clé pour des architectures flexibles et évolutives
- Factory Method & Abstract Factory sont des patterns de référence pour ce découplage
- Ils favorisent la modularité, la maintenabilité et l'utilisation de familles de produits cohérentes

2-3-1-1

Construction d'objets complexes pas à pas avec le pattern Builder

- Création d'objets complexes aux nombreux attributs parfois optionnels
- Construit étape par étape, évitant des constructeurs longs ou nombreux
- Améliore la lisibilité et la flexibilité du code

2-3-1-2

Principe du pattern Builder

- Sépare construction et représentation d'un objet complexe
- Rôles principaux :
 - **Builder** : interface ou classe abstraite avec étapes de construction
 - **ConcreteBuilder** : implémente Builder, assemble le produit
 - **Director** (optionnel) : orchestre les étapes via Builder
 - **Produit (Product)** : objet complexe construit
- Le client construit l'objet sans connaître l'assemblage interne

Exemple simple : classe Pizza

```
class Pizza {  
    private String size;  
    private boolean cheese;  
    private boolean pepperoni;  
    private boolean bacon;  
  
    private Pizza(PizzaBuilder builder) {  
        this.size = builder.size;  
        this.cheese = builder.cheese;  
        this.pepperoni = builder.pepperoni;  
        this.bacon = builder.bacon;  
    }  
  
    public static class PizzaBuilder {  
        private String size;  
        private boolean cheese;  
        private boolean pepperoni;  
        private boolean bacon;  
  
        public PizzaBuilder(String size) {  
            this.size = size;  
        }  
  
        public PizzaBuilder addCheese() {  
            this.cheese = true;  
        }  
    }  
}
```

```
    public PizzaBuilder addPepperoni() {
        this.pepperoni = true;
        return this;
    }

    public PizzaBuilder addBacon() {
        this.bacon = true;
        return this;
    }

    public Pizza build() {
        return new Pizza(this);
    }
}

@Override
public String toString() {
    return "Pizza [size=" + size + ", cheese=" + cheese + ", pepperoni=" + pepperoni + ", bacon=" + bacon + "];"
}

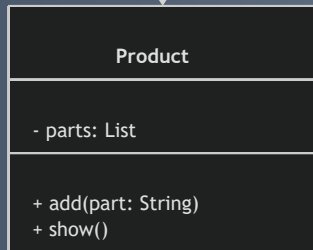
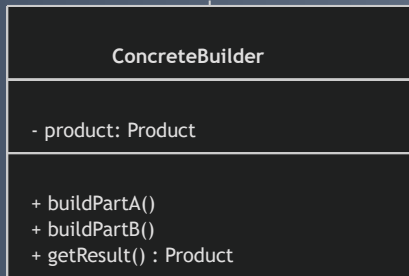
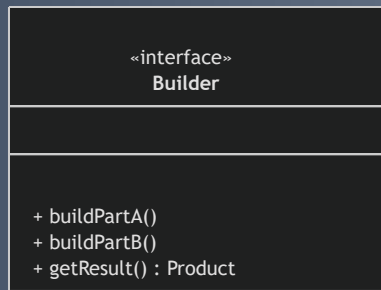
}
```

2-3-1-4

Utilisation de PizzaBuilder (suite)

```
public class Main {  
    public static void main(String[] args) {  
        Pizza pizza = new Pizza.PizzaBuilder("Large")  
            .addCheese()  
            .addPepperoni()  
            .build();  
        System.out.println(pizza);  
    }  
}
```

- Syntaxe fluide (fluent interface)
- Construction explicite et progressive



2-3-1-6

Bénéfices du pattern Builder

- Construction progressive et contrôlée d'objets complexes
- Lisibilité améliorée grâce au « fluent interface »
- Facilite la création d'objets immuables avec de nombreux paramètres
- Réduction de la surcharge de constructeurs
- Possibilité de créer plusieurs représentations d'un même produit

2-3-1-7

Contextes typiques d'utilisation

- Objets avec beaucoup d'attributs optionnels (ex : configuration, objets graphiques)
- Création de structures immuables requérant une configuration précise
- Fabrication passante par plusieurs étapes indépendantes

2-3-1-8

Sources utilisées

- Refactoring Guru, "Builder Design Pattern", refactoring.guru/design-patterns/builder
- Wikipedia, "Builder pattern", en.wikipedia.org/wiki/Builder_pattern
- Martin Fowler, "Builder", martinfowler.com/ieeeSoftware/Builder.pdf

Conclusion

Le pattern Builder permet de maîtriser la construction d'objets complexes

- Mécanisme simple et puissant
- Améliore la clarté du code
- Adapté aux contextes nécessitant flexibilité et robustesse à l'instanciation

2-3-2-1

Pattern Builder : Avantages et cas d'utilisation

- Facilite la construction d'objets complexes
- Gère de nombreux paramètres optionnels
- Offre flexibilité, clarté et maintenabilité du code

Avantages du pattern Builder

1. Gestion claire des objets complexes

1. Évite les constructeurs avec trop de paramètres ("constructor telescoping")
2. Construction étape par étape

2. Fluent interface

1. Syntaxe lisible et proche du langage naturel
2. Code client plus compréhensible

3. Création d'objets immuables

1. Construction encapsulée
2. Objet final non modifiable

2-3-2-3

Avantages (suite)

4. **Séparation construction / représentation**

1. Logique de construction distincte de la représentation finale
2. Possibilité de variantes différentes via plusieurs builders

5. **Flexibilité et extension aisée**

1. Ajout ou modification d'attributs sans impacter le client
2. Adaptable aux évolutions du besoin

2-3-2-4

Cas d'utilisation typiques

a) Objets avec de nombreux paramètres optionnels

- Ex : classe `Pizza`, `Voiture`, `Document`

b) Création d'objets immuables

- Ex : Java `StringBuilder`, objets `java.time`

c) Objets composés ou hiérarchiques

- Ex : interfaces utilisateur, documents XML/JSON

d) Processus de construction en étapes contrôlées

- Ex : construction d'une maison (fondations, murs, toiture)

2-3-2-5

Exemple simplifié : Configuration d'une voiture

```
public class Car {  
    private String engine;  
    private int seats;  
    private boolean GPS;  
  
    private Car(CarBuilder builder) {  
        this.engine = builder.engine;  
        this.seats = builder.seats;  
        this.GPS = builder.GPS;  
    }  
}
```

```
public static class CarBuilder {  
    private String engine;  
    private int seats;  
    private boolean GPS;  
  
    public CarBuilder engine(String engine) {  
        this.engine = engine;  
        return this;  
    }  
  
    public CarBuilder seats(int seats) {  
        this.seats = seats;  
        return this;  
    }  
  
    public CarBuilder GPS(boolean GPS) {  
        this.GPS = GPS;  
        return this;  
    }  
  
    public Car build() {  
        return new Car(this);  
    }  
}
```

```
// Utilisation
Car car = new Car.CarBuilder()
    .engine("V8")
    .seats(4)
    .GPS(true)
    .build();
```


2-3-2-7

Sources

- Refactoring Guru – Builder design pattern
- Wikipedia – Builder pattern
- Baeldung – Builder Design Pattern in Java

Conclusion

Le pattern Builder :

- Solution élégante pour objets complexes
- Code clair, modulaire et maintenable
- Particulièrement efficace avec de multiples paramètres ou étapes de construction

2-4-1-1

Clonage d'objets avec le pattern Prototype

Introduction

- Le pattern **Prototype** crée des objets en copiant des instances existantes.
- Utile pour créer rapidement des objets lourds ou quand les classes ne sont pas connues à l'avance.

2-4-1-2

Principe du pattern Prototype

- Basé sur le clonage d'un objet modèle (prototype).
- Définit une interface commune avec une méthode `clone()`.

Points clés

- **Prototype** : interface ou classe abstraite avec `clone()`.
- **ConcretePrototype** : implémente le clonage.
- Le client crée un objet en clonant, sans instancier une classe.

2-4-1-3

Clonage superficiel vs profond

Type de clonage	Description	Impact
Clonage superficiel	Copie l'objet sans copier les objets référencés	Références partagées, modifications sur un objet peuvent affecter l'autre
Clonage profond	Copie aussi les objets référencés	Objets indépendants, évite les effets de bord

- Le clonage profond est souvent nécessaire.

Exemple (extrait clé)

```
public abstract class Prototype implements Cloneable {
    public Prototype clone() throws CloneNotSupportedException {
        return (Prototype) super.clone();
    }
}

public class Person extends Prototype {
    private String name;
    private int age;
    private Address address;

    // Clonage profond
    @Override
    public Person clone() throws CloneNotSupportedException {
        Person cloned = (Person) super.clone();
        cloned.address = address.clone(); // deep copy
        return cloned;
    }
}
```

- **Person** clone son **address** pour un clonage profond.

Exemple (suite)

```
public class Address implements Cloneable {
    private String city;
    private String street;

    @Override
    protected Address clone() throws CloneNotSupportedException {
        return (Address) super.clone();
    }
}

public class Main {
    public static void main(String[] args) throws CloneNotSupportedException {
        Address address = new Address("Paris", "Rue de Rivoli");
        Person original = new Person("Alice", 30, address);

        Person clone = original.clone();
        clone.address = new Address("Lyon", "Rue Saint-Jean"); // modifie clone sans impacter original
    }
}
```

- Le clone est modifiable sans altérer l'original grâce au clonage profond.

Diagramme illustrant le pattern Prototype

2-4-1-7

Avantages

- Évite la complexité d'instanciation avec `new`, utile pour objets lourds.
- Clonage sans connaître la classe exacte des objets.
- Permet réutilisation de prototypes configurés.
- Réduit le coût de création quand le clonage est plus efficace.

2-4-1-8

Cas d'utilisation

- Création d'objets complexes ou coûteux à instancier.
- Systèmes avec instanciation dynamique (éditeurs graphiques, frameworks).
- Personnalisation basée sur des prototypes déjà configurés.

2-4-1-9

Sources utilisées

- Refactoring Guru, "Prototype design pattern", <https://refactoring.guru/design-patterns/prototype>
- Wikipedia, "Prototype pattern", https://en.wikipedia.org/wiki/Prototype_pattern
- Gamma et al., "Design Patterns: Elements of Reusable Object-Oriented Software", Addison-Wesley, 1994.

Conclusion

Le pattern Prototype offre une méthode élégante pour cloner des objets existants.

- Convient aux objets complexes, configurés ou dynamiques.
- Simplifie le code client.
- Améliore la performance de création d'objets.

2-4-2-1

Gestion de l'état initial des objets avec le pattern Prototype

Introduction

- La gestion de l'état initial dans **Prototype** est cruciale pour garantir la cohérence des objets clonés.
- Elle évite les erreurs liées au partage ou à la réutilisation inappropriée des données.

2-4-2-2

Problématique de l'état initial lors du clonage

- Clonage peut provoquer :
 - Partage d'état entre original et copie → effets secondaires non désirés.
 - Nécessité de réinitialiser ou modifier l'état dans le clone pour un nouveau contexte.
- Important avec attributs mutables et ressources externes (fichiers, connexions).

2-4-2-3

Approches pour gérer l'état initial (1/2)

1. Clonage profond (Deep Clone)

- Copie indépendante de tous les objets référencés.
- Garantit que modifications du clone n'impactent pas l'original.

2. Personnalisation post-clonage

- Adapter certains attributs après clonage (ex : identifiants uniques).
- Par constructeur de copie, méthode `init` ou `reset`, ou `clone()` personnalisée.

2-4-2-4

Approches pour gérer l'état initial (2/2)

3. Immutabilité partielle

- Rendre objets partiellement ou totalement immuables.
- Réduit les problèmes liés à l'état partagé.

Exemple concret : Classe Person

```
public class Person implements Cloneable {  
    private String id;  
    private String name;  
    private Address address;  
  
    public Person(String id, String name, Address address) {  
        this.id = id;  
        this.name = name;  
        this.address = address;  
    }  
  
    @Override  
    protected Person clone() throws CloneNotSupportedException {  
        Person cloned = (Person) super.clone();  
        cloned.address = address.clone(); // deep copy  
        cloned.id = generateNewId();      // reset ID pour un nouvel état  
        return cloned;  
    }  
  
    private String generateNewId() {  
        return "ID-" + System.currentTimeMillis();  
    }  
}
```

2-4-2-6

Exemple concret : Classe Address

```
public class Address implements Cloneable {  
    private String city;  
    private String street;  
  
    public Address(String city, String street) {  
        this.city = city;  
        this.street = street;  
    }  
  
    @Override  
    protected Address clone() throws CloneNotSupportedException {  
        return (Address) super.clone();  
    }  
}
```


2-4-2-7

Utilisation de l'exemple

```
Person original = new Person("ID-123", "Alice", new Address("Paris", "Rue A"));
Person clone = original.clone();
System.out.println(original);
System.out.println(clone);
```

- La méthode `clone()` :
 - Réalise un clonage profond de `address`.
 - Réinitialise `id` afin d'assurer un état distinct du clone.

Diagramme – Gestion de l'état dans Prototype

2-4-2-9

Points clés à retenir

- Clonage = copier données + gérer finement l'état initial.
- Clonage profond prévient les erreurs liées aux références partagées.
- L'état initial du clone peut nécessiter réinitialisation ou adaptation selon contexte métier.
- Une logique claire de gestion d'état assure la robustesse du pattern Prototype.

2-4-2-10

Sources consultées

- Refactoring Guru, "Prototype Pattern — When and How to Use It"
- Oracle Java Tutorials, "Cloneable Interface"
- Wikipedia, "Prototype pattern"

Conclusion

La gestion soignée de l'état initial des objets clonés maximise les bénéfices du pattern Prototype en assurant la création d'instances fiables, distinctes et adaptées à leurs usages spécifiques.

3-1-1-1

Intégration de composants tiers avec le pattern Adapter

Introduction

- Réutilisation difficile des composants tiers à cause d'incompatibilités d'interfaces
- **Pattern Adapter** : adapte l'interface d'un composant existant à celle attendue par le client
- Facilite l'interopérabilité sans modifier les composants originaux

3-1-1-2

Principe du pattern Adapter

- Agit comme un **pont** entre deux interfaces incompatibles
- Permet au client d'utiliser un composant tiers avec une interface différente
- Fournit une interface intermédiaire conforme aux attentes du client

Rôles

- **Client** : utilise l'interface cible
- **Target (interface cible)** : attendue par le client
- **Adaptee** : composant tiers avec interface incompatible
- **Adapter** : implémente Target, traduit les appels vers Adaptee

Exemple concret (1/3)

```
interface MediaPlayer {
    void play(String audioType, String fileName);
}

interface AdvancedMediaPlayer {
    void playVlc(String fileName);
    void playMp4(String fileName);
}

class VlcPlayer implements AdvancedMediaPlayer {
    public void playVlc(String fileName) {
        System.out.println("Playing vlc file: " + fileName);
    }
    public void playMp4(String fileName) { }
}

class Mp4Player implements AdvancedMediaPlayer {
    public void playVlc(String fileName) { }
    public void playMp4(String fileName) {
        System.out.println("Playing mp4 file: " + fileName);
    }
}
```

Exemple concret (2/3)

```
class MediaAdapter implements MediaPlayer {
    AdvancedMediaPlayer advancedMusicPlayer;

    public MediaAdapter(String audioType) {
        if(audioType.equalsIgnoreCase("vlc")) {
            advancedMusicPlayer = new VlcPlayer();
        } else if(audioType.equalsIgnoreCase("mp4")) {
            advancedMusicPlayer = new Mp4Player();
        }
    }

    public void play(String audioType, String fileName) {
        if(audioType.equalsIgnoreCase("vlc")) {
            advancedMusicPlayer.playVlc(fileName);
        } else if(audioType.equalsIgnoreCase("mp4")) {
            advancedMusicPlayer.playMp4(fileName);
        }
    }
}
```

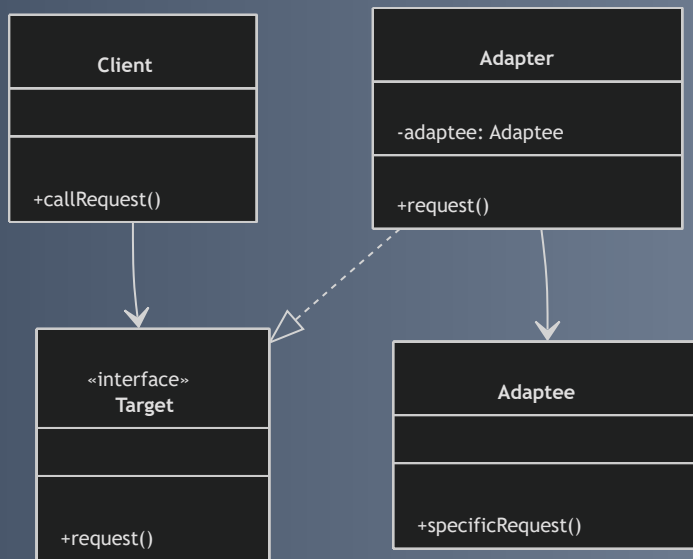

Exemple concret (3/3)

```
class AudioPlayer implements MediaPlayer {
    MediaAdapter mediaAdapter;

    public void play(String audioType, String fileName) {
        if(audioType.equalsIgnoreCase("mp3")) {
            System.out.println("Playing mp3 file: " + fileName);
        } else if(audioType.equalsIgnoreCase("vlc") || audioType.equalsIgnoreCase("mp4")) {
            mediaAdapter = new MediaAdapter(audioType);
            mediaAdapter.play(audioType, fileName);
        } else {
            System.out.println("Invalid media. " + audioType + " format not supported");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        AudioPlayer player = new AudioPlayer();
        player.play("mp3", "song.mp3");
        player.play("mp4", "video.mp4");
        player.play("vlc", "movie.vlc");
        player.play("avi", "file.avi");
    }
}
```

Diagramme illustrant le pattern Adapter



3-1-1-7

Avantages du pattern Adapter dans l'intégration de composants tiers

- **Réutilisation sans modification** : utilise des composants inaltérables
- **Flexibilité** : plusieurs interfaces adaptées à une interface commune
- **Découplage** : client indépendant des classes concrètes
- **Gestion progressive de l'évolution** : intègre de nouvelles bibliothèques sans refonte

3-1-1-8

Cas pratiques d'utilisation

- Intégration de bibliothèques externes aux API différentes
- Migration progressive d'un ancien système vers un nouveau
- Adaptation d'interfaces entre modules développés par des équipes différentes

3-1-1-9

Conclusion

- Le pattern Adapter est clé dans les architectures modulaires
- Permet coexistence et interopérabilité de composants variés
- Simplifie l'intégration de composants tiers sans toucher au système existant

3-1-2-1

Interopérabilité entre interfaces incompatibles avec le pattern Adapter

Introduction

- Les interfaces différentes empêchent souvent la collaboration entre composants.
- Le **pattern Adapter** permet une interopérabilité sans modifier les composants.
- Il fournit une couche intermédiaire qui traduit une interface en une autre.

3-1-2-2

Comprendre l'Interopérabilité via l'Adapter

- L'Adapter agit comme un traducteur entre interfaces incompatibles :
 - **Reçoit les appels clients** selon l'interface attendue (Target).
 - **Traduit ces appels** vers une interface différente (Adaptee).
 - Le client interagit avec l'Adapter comme avec la Target.
- Pas besoin de modifier le client ou le composant tiers pour assurer la compatibilité.

3-1-2-3

Formes d'Adapter

- **Adapter par composition (objet) :**
 - L'adapter contient une instance de l'adaptee et délègue les appels.
 - Méthode recommandée et la plus flexible.
- **Adapter par héritage (classe) :**
 - L'adapter hérite de l'adaptee et implémente l'interface target.

3-1-2-4

Exemple : Intégration entre interfaces incompatibles

```
// Interface américaine
interface USPowerPlug {
    void connect();
}

// Interface européenne
interface EUPowerPlug {
    void connectEuropean();
}

// Composant existant - prise européenne
class EUPowerDevice implements EUPowerPlug {
    public void connectEuropean() {
        System.out.println("Connected to European power plug.");
    }
}
```

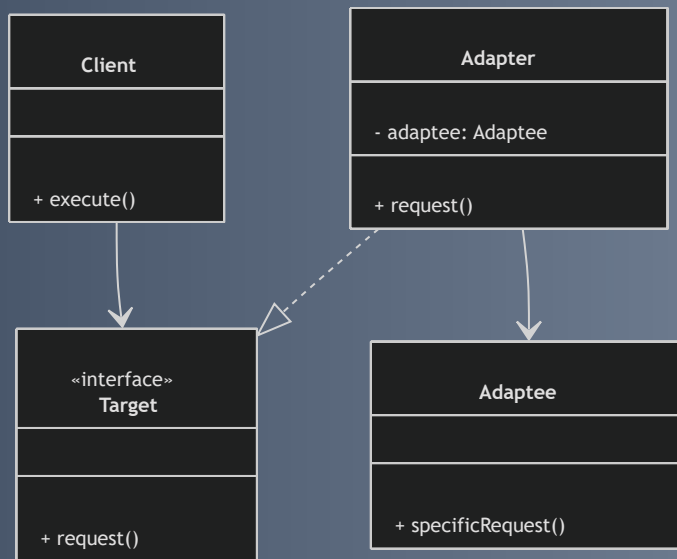
```
// Adapter pour rendre compatible une prise EU à une prise US
class PowerPlugAdapter implements USPowerPlug {
    private EUPowerPlug euPlug;

    public PowerPlugAdapter(EUPowerPlug euPlug) {
        this.euPlug = euPlug;
    }

    public void connect() {
        System.out.print("Adapting US plug to European plug... ");
        euPlug.connectEuropean();
    }
}

// Client utilisant USPowerPlug
public class Client {
    public static void main(String[] args) {
        EUPowerPlug euDevice = new EUPowerDevice();
        USPowerPlug adapter = new PowerPlugAdapter(euDevice);
        adapter.connect();
    }
}
```

Diagramme illustrant l'interopérabilité avec Adapter



3-1-2-6

Avantages clés dans l'Interopérabilité

- **Préservation des classes existantes** sans modification.
- **Facilite l'intégration progressive** de nouveaux composants.
- **Renforce la modularité** et la séparation des responsabilités.
- **Permet l'utilisation de composants tiers ou legacy** malgré des interfaces différentes.

3-1-2-7

Sources utilisées

- Refactoring Guru, "Adapter Design Pattern", <https://refactoring.guru/design-patterns/adapter>
- Baeldung, "Adapter Pattern in Java", <https://www.baeldung.com/java-adapter-pattern>
- Wikipedia, "Adapter Pattern", https://en.wikipedia.org/wiki/Adapter_pattern

Conclusion

Le pattern Adapter est un outil puissant garantissant :

- L'interopérabilité entre systèmes à interfaces incompatibles,
- Une intégration propre et maintenable,
- Une meilleure modularité dans un environnement logiciel hétérogène.

3-2-1-1

Extension dynamique des comportements sans héritage : le pattern Decorator

- Permet d'ajouter ou modifier un comportement d'un objet **à l'exécution**
- Evite l'héritage statique et la multiplication des sous-classes
- Favorise la flexibilité et l'adaptabilité en conception orientée objet

3-2-1-2

Principe du pattern Decorator

- Un **décorateur** enveloppe un objet existant (composant)
- Le décorateur et le composant ont la **même interface**
- Le décorateur ajoute ou modifie des fonctionnalités avant/après délégation
- Décorateurs peuvent s'imbriquer en couches successives

3-2-1-3

Caractéristiques principales

- Interface commune entre décorateur et objet décoré
- Référence interne du décorateur vers l'objet décoré
- Empilement possible de plusieurs décorateurs pour combiner des comportements
- Évite l'explosion du nombre de sous-classes liée aux combinaisons

3-2-1-4

Exemple concret : gestion dynamique de formats de texte

```
interface Text {  
    String getContent();  
}  
  
class PlainText implements Text {  
    private String content;  
    public PlainText(String content) { this.content = content; }  
    @Override  
    public String getContent() { return content; }  
}  
  
abstract class TextDecorator implements Text {  
    protected Text decoratedText;  
    public TextDecorator(Text decoratedText) { this.decoratedText = decoratedText; }  
}
```

```
class UnderlineDecorator extends TextDecorator {
    public UnderlineDecorator(Text decoratedText) { super(decoratedText); }
    @Override
    public String getContent() {
        return "<u>" + decoratedText.getContent() + "</u>";
    }
}

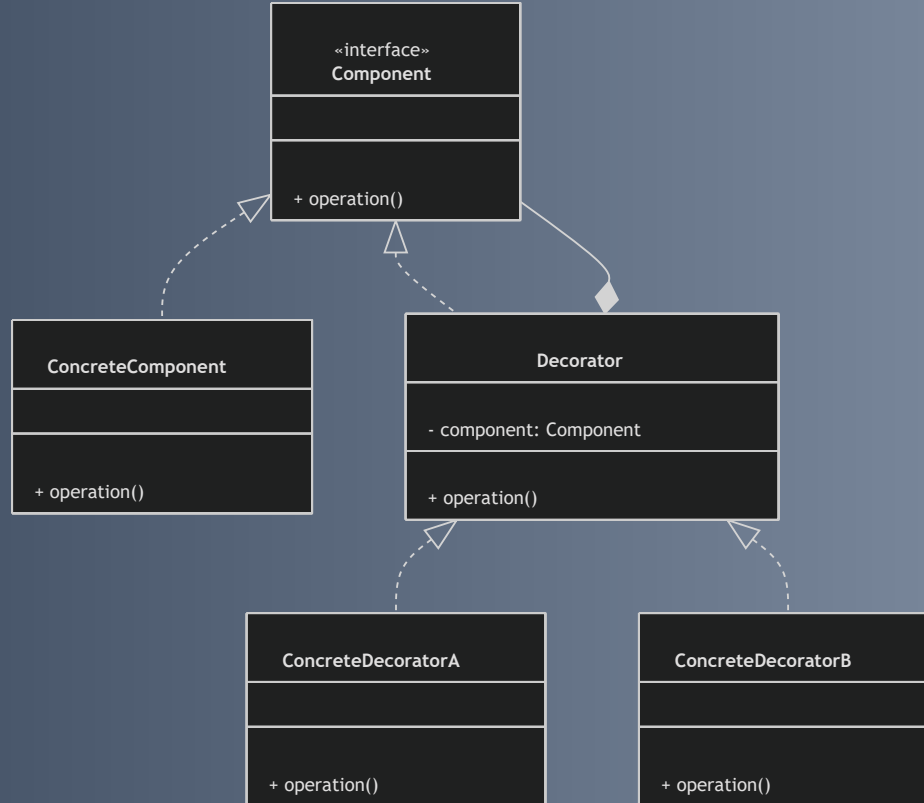
class HighlightDecorator extends TextDecorator {
    public HighlightDecorator(Text decoratedText) { super(decoratedText); }
    @Override
    public String getContent() {
        return "<mark>" + decoratedText.getContent() + "</mark>";
    }
}
```

3-2-1-5

Exemple d'utilisation : empilement des décorateurs

```
public class Main {  
    public static void main(String[] args) {  
        Text simpleText = new PlainText("Hello, World!");  
  
        Text underlined = new UnderlineDecorator(simpleText);  
        Text highlighted = new HighlightDecorator(underlined);  
  
        System.out.println("Simple: " + simpleText.getContent());  
        System.out.println("Underlined: " + underlined.getContent());  
        System.out.println("Highlighted and Underlined: " + highlighted.getContent());  
    }  
}
```

Structure du pattern Decorator



3-2-1-7

Avantages du pattern Decorator

- Extension dynamique des comportements à l'exécution
- Composition souple par empilement de décorateurs
- Évite multi-héritage et explosion de sous-classes
- Respecte le principe de responsabilité unique
- Facilite la gestion fine et modulaire des comportements

3-2-1-8

Cas d'usage fréquents

- Interfaces graphiques : bordures, scrollbars, effets visuels
- Flux de données : compression, chiffrement, logging
- Ajout de fonctionnalités transverses sans modifier les classes sources

3-2-1-9

Sources utilisées

- Refactoring Guru, "Decorator Pattern"
- Oracle Java Tutorials, "Decorator Pattern"
- Gamma et al., "Design Patterns: Elements of Reusable Object-Oriented Software", 1994

Conclusion

Le pattern Decorator est une alternative puissante à l'héritage :

il permet d'enrichir dynamiquement des objets sans rigidité ni complexité excessive, garantissant un code propre, flexible et facile à maintenir.

3-3-1-1

Simplification d'interfaces complexes via le pattern Facade

- Plusieurs sous-systèmes complexes rendent l'accès lourd et sujet à erreurs
- Le **pattern Facade** : une interface unique et simplifiée
- Objectif : masquer la complexité et faciliter l'utilisation

3-3-1-2

Principes du pattern Facade

- **Point d'entrée unique** vers plusieurs classes
- Encapsulation des interactions et dépendances
- Bénéfices :
 - Abstraction réduisant la charge cognitive
 - Découplage client-sous-systèmes
 - Flexibilité interne sans impacter le client
- Différence avec l'adaptateur : simplification d'accès, pas transformation d'interface

Exemple : système de gestion multimédia avec plusieurs sous-systèmes

Sous-systèmes :

```
class AudioSystem {  
    public void playAudio(String file) { ... }  
}  
class VideoSystem {  
    public void playVideo(String file) { ... }  
}  
class NetworkSystem {  
    public void connect(String url) { ... }  
}  
class CacheSystem {  
    public void cacheMedia(String file) { ... }  
}
```

Exemple : façade unifiée multimédia

```
class MediaFacade {  
    private AudioSystem audio = new AudioSystem();  
    private VideoSystem video = new VideoSystem();  
    private NetworkSystem network = new NetworkSystem();  
    private CacheSystem cache = new CacheSystem();  
  
    public void playMedia(String url, String fileType, String fileName) {  
        network.connect(url);  
        cache.cacheMedia(fileName);  
        if (fileType.equalsIgnoreCase("audio")) {  
            audio.playAudio(fileName);  
        } else if (fileType.equalsIgnoreCase("video")) {  
            video.playVideo(fileName);  
        } else {  
            System.out.println("Format non supporté");  
        }  
    }  
}
```

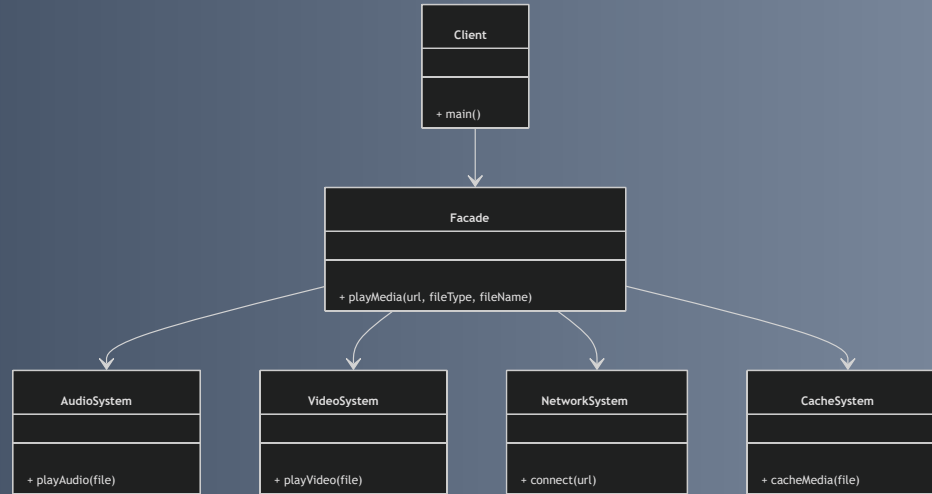
3-3-1-5

Exemple : utilisation cliente

```
public class Client {  
    public static void main(String[] args) {  
        MediaFacade mediaFacade = new MediaFacade();  
        mediaFacade.playMedia("http://media.example.com", "audio", "song.mp3");  
        mediaFacade.playMedia("http://media.example.com", "video", "movie.mp4");  
    }  
}
```

3-3-1-6

Diagramme illustrant le pattern Facade



3-3-1-7

Avantages du pattern Facade

- **Simplification** : interface plus simple et cohérente
- **Encapsulation** : cache les détails complexes des sous-systèmes
- **Découplage** : minimise la dépendance du client aux classes internes
- **Évolutivité** : structure interne modifiable sans impact client

3-3-1-8

Cas d'utilisation typiques

- Bibliothèques complexes exposant plusieurs APIs
- Systèmes modulaires avec divers sous-systèmes
- Réduction de la complexité dans les architectures distribuées

3-3-1-9

Sources utilisées

- Refactoring Guru, "Facade design pattern"
- Baeldung, "Facade Pattern in Java"
- Gamma et al., "Design Patterns", Addison-Wesley, 1994

3-3-1-10

Conclusion

Le pattern Facade est une solution élégante pour orchestrer la complexité via une interface simplifiée, améliorant ainsi :

- La maintenabilité
- La clarté du code
- L'expérience développeur face à des architectures riches en composants

3-4-1-1

Gestion d'arborescences et structures récursives avec le pattern Composite

- Les structures hiérarchiques (arbres, graphes) sont difficiles à manipuler uniformément.
- Le **pattern Composite** permet de traiter objets simples et compositions de manière homogène.

3-4-1-2

Principe du pattern Composite

- Hiérarchie arborescente d'objets :
 - **Feuilles** : objets simples sans enfants.
 - **Composites** : objets contenant des composants (feuilles ou composites).
- Interface commune définie pour uniformiser les opérations.
- Simplifie la manipulation récursive des structures complexes.

3-4-1-3

Structure du pattern Composite

Élément	Description
Component	Interface abstraite commune.
Leaf	Objet simple (feuille) sans enfants.
Composite	Objet contenant d'autres composants (feuilles ou composites).

Exemple : gestion de dossiers et fichiers

```
interface FileSystemNode {  
    void ls();  
}  
  
class File implements FileSystemNode {  
    private String name;  
  
    public File(String name) {  
        this.name = name;  
    }  
  
    @Override  
    public void ls() {  
        System.out.println("File: " + name);  
    }  
}
```

```
class Directory implements FileSystemNode {
    private String name;
    private List<FileSystemNode> children = new ArrayList<>();

    public Directory(String name) {
        this.name = name;
    }

    public void add(FileSystemNode node) {
        children.add(node);
    }

    public void remove(FileSystemNode node) {
        children.remove(node);
    }

    @Override
    public void ls() {
        System.out.println("Directory: " + name);
        for (FileSystemNode child : children) {
            child.ls();
        }
    }
}
```

--> Suite -->

```
public class Client {  
    public static void main(String[] args) {  
        Directory root = new Directory("root");  
        root.add(new File("file1.txt"));  
  
        Directory subDir = new Directory("subdir");  
        subDir.add(new File("file2.txt"));  
  
        root.add(subDir);  
  
        root.ls();  
    }  
}
```

3-4-1-5

Sortie attendue de l'exemple

```
Directory: root  
File: file1.txt  
Directory: subdir  
File: file2.txt
```

Diagramme du pattern Composite

3-4-1-7

Avantages du pattern Composite

- **Uniformité** pour manipuler objets simples et composés via une interface unique.
- **Extensibilité** : nouveaux composants sans modifier le client.
- Simplicité des **algorithmes récursifs** sur arbres.
- Modélisation naturelle d'objets hiérarchiques (documents, UI, fichiers).

3-4-1-8

Cas d'utilisation courants

- Gestion de fichiers et dossiers.
- Arborescences graphiques (scènes 3D, interfaces).
- Documents structurés (HTML, XML).
- Hiérarchies organisationnelles.

3-4-1-9

Sources utilisées

- Refactoring Guru, "Composite pattern", <https://refactoring.guru/design-patterns/composite>
- Baeldung, "Composite design pattern in Java", <https://www.baeldung.com/java-composite-pattern>
- Gamma et al., "Design Patterns: Elements of Reusable Object-Oriented Software", Addison-Wesley, 1994

Conclusion

- Le pattern Composite uniformise l'interaction avec objets simples ou composés.
- Il simplifie la gestion des structures récursives complexes.
- Permet un code plus clair, extensible et robuste.

3-5-1-1

Contrôle d'accès avec le pattern Proxy

- Contrôle l'accès à un objet via un substitut (Proxy).
- Utile pour :
 - Sécurité
 - Gestion de la charge
 - Contrôle d'accès
- Cache la complexité au client.

3-5-1-2

Principe du pattern Proxy

- Proxy implémente la même interface que l'objet réel (*subject*).
- Intercepte les appels pour vérifier permissions avant délégation.

Rôles :

- **Subject** : interface commune.
- **RealSubject** : objet réel.
- **Proxy** : gardien, contrôle l'accès et peut ajouter des comportements.

Exemple : service bancaire avec contrôle d'accès

```
interface BankService {  
    void withdraw(String userRole, double amount);  
}  
  
class RealBankService implements BankService {  
    private double balance = 1000;  
  
    @Override  
    public void withdraw(String userRole, double amount) {  
        if(amount ≤ balance) {  
            balance -= amount;  
            System.out.println("Retrait de " + amount + " effectué. Solde restant : " + balance);  
        } else {  
            System.out.println("Fonds insuffisants.");  
        }  
    }  
}
```

--> Suite -->

```
class BankServiceProxy implements BankService {
    private RealBankService realService = new RealBankService();

    @Override
    public void withdraw(String userRole, double amount) {
        if(userRole.equalsIgnoreCase("admin") || userRole.equalsIgnoreCase("user")) {
            realService.withdraw(userRole, amount);
        } else {
            System.out.println("Accès refusé: rôle utilisateur non autorisé.");
        }
    }
}

public class Client {
    public static void main(String[] args) {
        BankService bankService = new BankServiceProxy();

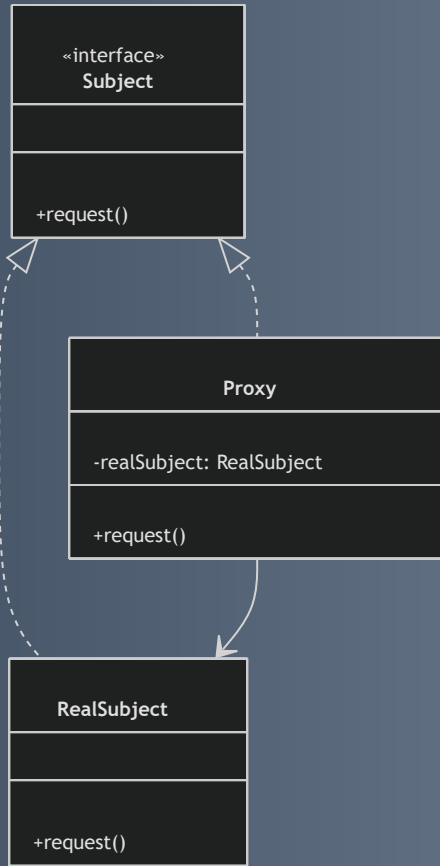
        System.out.println("Tentative par admin :");
        bankService.withdraw("admin", 200);

        System.out.println("Tentative par invité :");
        bankService.withdraw("guest", 200);
    }
}
```

3-5-1-4

Sortie attendue de l'exemple

```
Tentative par admin :  
Retrait de 200.0 effectué. Solde restant : 800.0  
Tentative par invité :  
Accès refusé: rôle utilisateur non autorisé.
```

3-5-1-6

Avantages du pattern Proxy pour le contrôle d'accès

- Séparation claire des préoccupations (sécurité dans le proxy).
- Centralisation des contrôles sans modifier l'objet réel.
- Flexibilité pour adapter la logique d'accès sans modifier les clients.
- Peut combiner d'autres responsabilités : logging, cache, lazy loading.

3-5-1-7

Usages courants

- Contrôle d'accès et authentification.
- Proxy distant (Remote Proxy) pour accès réseau sécurisé.
- Proxy virtuel pour chargement différé d'objets lourds.
- Proxy de protection en architectures distribuées.

3-5-1-8

Sources utilisées

- Refactoring Guru, "Proxy design pattern", <https://refactoring.guru/design-patterns/proxy>
- Baeldung, "Proxy Pattern in Java", <https://www.baeldung.com/java-proxy-pattern>
- Gamma et al., "Design Patterns: Elements of Reusable Object-Oriented Software", Addison-Wesley, 1994.

Conclusion

Le pattern Proxy est une solution souple et puissante pour

- Contrôler l'accès à des ressources sensibles
- Intégrer une couche intermédiaire qui protège et régule l'usage des objets réels

3-5-2-1

Lazy loading avec le pattern Proxy

Introduction

- Le **lazy loading** retarde l'initialisation d'un objet coûteux jusqu'à son usage effectif.
- Le **proxy virtuel** implémente cette stratégie : il intercepte les appels et crée l'instance lourde uniquement à la demande.

3-5-2-2

Principe du lazy loading via Proxy

- Le **proxy virtuel** est un substitut à l'objet réel.
- Il contient une référence non initialisée à cet objet coûteux.
- À la première invocation, il crée l'objet réel puis délègue l'appel.
- Optimise mémoire et temps de démarrage.

Exemple : lecture d'une image lourde

```
// Interface commune
interface Image {
    void display();
}

// Objet réel lourd
class RealImage implements Image {
    private String filename;

    public RealImage(String filename) {
        this.filename = filename;
        loadFromDisk();
    }

    private void loadFromDisk() {
        System.out.println("Chargement de l'image depuis le disque : " + filename);
    }

    @Override
    public void display() {
        System.out.println("Affichage de l'image : " + filename);
    }
}
```

```
// Proxy virtuel implémentant le lazy loading
class ProxyImage implements Image {
    private RealImage realImage;
    private String filename;

    public ProxyImage(String filename) {
        this.filename = filename;
    }

    @Override
    public void display() {
        if (realImage == null) {
            realImage = new RealImage(filename); // Chargement différé
        }
        realImage.display();
    }
}

// Utilisation
public class Client {
    public static void main(String[] args) {
        Image image = new ProxyImage("photo.jpg");
        System.out.println("Image créée mais non chargée.");
        image.display(); // Chargement effectif ici
        image.display(); // Utilisation sans rechargement
    }
}
```


3-5-2-4

Sortie attendue de l'exemple

```
Image créée mais non chargée.  
Chargement de l'image depuis le disque : photo.jpg  
Affichage de l'image : photo.jpg  
Affichage de l'image : photo.jpg
```

3-5-2-6

Avantages du lazy loading avec Proxy

- **Amélioration des performances** par report des opérations coûteuses.
- **Réduction de la consommation mémoire** en évitant les chargements inutiles.
- **Transparence** : même interface que l'objet réel.
- **Gestion simplifiée** des objets volumineux ou sources lentes.

3-5-2-7

Cas d'usage fréquents

- Chargement d'images, documents, ou vidéos volumineux.
- Accès différé à bases de données ou fichiers distants.
- Instanciation d'objets lourds peu utilisés.

3-5-2-8

Sources utilisées

- Refactoring Guru, "Proxy design pattern" : <https://refactoring.guru/design-patterns/proxy>
- Baeldung, "Proxy Pattern in Java" : <https://www.baeldung.com/java-proxy-pattern>
- Gamma et al., "Design Patterns", Addison-Wesley, 1994

Conclusion

Le pattern Proxy, via son proxy virtuel, réalise un lazy loading simple et efficace.

Il optimise l'utilisation des ressources en adaptant l'instanciation des objets lourds aux besoins réels de l'application.

3-5-3-1

Logging transversal avec le pattern Proxy

Introduction

- Le **pattern Proxy** insère une couche entre client et objet réel.
- Permet d'ajouter des fonctionnalités transversales sans toucher au code métier.
- Usage fréquent : **logging transversal** (tracer appels, durée d'exécution, données).

3-5-3-2

Principe du logging transversal via Proxy

- Proxy agit comme un **aspect** autour de l'objet réel.
- **Interception** des appels de méthode.
- **Enregistrement** des infos : paramètres, temps, erreurs.
- **Délégation** à l'objet réel après logging.
- Respecte le principe de **séparation des préoccupations**.

3-5-3-3

Exemple : Proxy simple pour logging

```
interface Service {  
    void process(String request);  
}  
  
class RealService implements Service {  
    @Override  
    public void process(String request) {  
        System.out.println("Traitement de : " + request);  
    }  
}
```

--> Suite -->

```
class LoggingProxy implements Service {
    private Service realService;
    public LoggingProxy(Service realService) {
        this.realService = realService;
    }
    @Override
    public void process(String request) {
        System.out.println("[LOG] Appel de process avec paramètres : " + request);
        long start = System.currentTimeMillis();
        realService.process(request);
        long end = System.currentTimeMillis();
        System.out.println("[LOG] Durée d'exécution : " + (end - start) + " ms");
    }
}

public class Client {
    public static void main(String[] args) {
        Service service = new LoggingProxy(new RealService());
        service.process("exemple de requête");
    }
}
```


3-5-3-4

Exemple de sortie console

```
[LOG] Appel de process avec paramètres : exemple de requête  
Traitement de : exemple de requête  
[LOG] Durée d'exécution : 10 ms
```

3-5-3-6

Avantages du pattern Proxy pour le logging

- **Non-intrusif** : objet réel inchangé.
- **Réutilisable** : s'applique à plusieurs services.
- **Extensible** : ajoute/évolue sans impacter la logique métier.
- **Maintenance simplifiée** : code de logging centralisé.

3-5-3-7

Cas d'usage typiques

- Traçabilité dans architectures distribuées.
- Debugging et surveillance en production.
- Mesure de performances (profiling).
- Audits de sécurité.

3-5-3-8

Sources utilisées

- Refactoring Guru, "Proxy pattern" : <https://refactoring.guru/design-patterns/proxy>
- Baeldung, "Proxy Pattern in Java" : <https://www.baeldung.com/java-proxy-pattern>
- Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994

Conclusion

Le pattern Proxy permet d'introduire facilement un **logging transversal**

- Ajoute une couche de traçabilité à n'importe quel service
- Respecte la **séparation des responsabilités**
- Maintient un code métier propre et clair

4-1-1-1

Gestion des événements avec le pattern Observer et Event Bus

- Communication entre composants basée sur événements
- Émetteurs envoient des notifications
- Récepteurs reçoivent et traitent ces notifications
- Pattern **Observer** : gestion décentralisée d'événements
- **Event Bus** : extension moderne asynchrone et scalable

4-1-1-2

Le pattern Observer : principes clés

- Relation **un-à-plusieurs** entre objets
- **Sujet (Subject)** maintient une liste d'observateurs
- Lors d'un événement, le sujet notifie tous les observateurs
- Observateurs réagissent selon leur propre logique
- Découplage fort entre émetteur et récepteurs

4-1-1-3

Exemple Java : interfaces Observer et Subject

```
interface Observer {  
    void update(float temperature);  
}  
  
interface Subject {  
    void registerObserver(Observer o);  
    void removeObserver(Observer o);  
    void notifyObservers();  
}
```

4-1-1-4

Exemple : sujet concret WeatherData

```
import java.util.ArrayList;
import java.util.List;

class WeatherData implements Subject {
    private List<Observer> observers = new ArrayList<>();
    private float temperature;

    @Override
    public void registerObserver(Observer o) {
        observers.add(o);
    }
}
```



```
@Override
public void removeObserver(Observer o) {
    observers.remove(o);
}

@Override
public void notifyObservers() {
    for (Observer o : observers) {
        o.update(temperature);
    }
}

public void setTemperature(float temperature) {
    this.temperature = temperature;
    notifyObservers();
}
}
```

4-1-1-5

Exemple : observateur concret CurrentConditionsDisplay

```
class CurrentConditionsDisplay implements Observer {  
    private float temperature;  
  
    @Override  
    public void update(float temperature) {  
        this.temperature = temperature;  
        display();  
    }  
  
    public void display() {  
        System.out.println("Température actuelle : " + temperature + "°C");  
    }  
}
```

Exemple utilisation du pattern Observer

```
public class Client {  
    public static void main(String[] args) {  
        WeatherData weatherData = new WeatherData();  
  
        CurrentConditionsDisplay display = new CurrentConditionsDisplay();  
        weatherData.registerObserver(display);  
  
        weatherData.setTemperature(25.3f);  
        weatherData.setTemperature(27.8f);  
    }  
}
```

Sortie console :

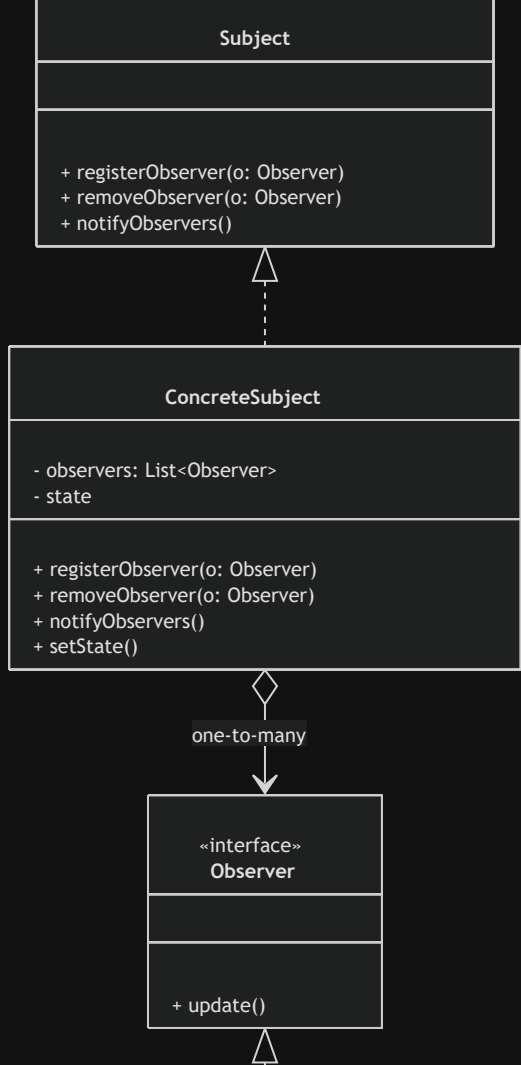
Température actuelle : 25.3°C

Température actuelle : 27.8°C

4-1-1-7

Event Bus

- Système centralisé de publication/abonnement
- Découple complètement émetteurs et récepteurs
- Gestion d'événements diversifiés et asynchrones
- Scalabilité via mécanisme basé sur des topics
- Exemples : Google Guava EventBus, Vert.x Event Bus



4-1-1-9

Avantages du pattern Observer et Event Bus

- **Découplage fort** entre émetteurs et récepteurs
- **Extensibilité** via ajout/suppression dynamique d'observateurs
- **Flexibilité** pour gérer des événements variés
- Convient aux architectures événementielles modernes :
microservices, interfaces utilisateur, IoT

4-1-1-10

Sources

- Refactoring Guru, "Observer pattern", <https://refactoring.guru/design-patterns/observer>
- Baeldung, "Observer pattern in Java", <https://www.baeldung.com/java-observer-pattern>
- Google Guava EventBus documentation, <https://github.com/google/guava/wiki/EventBusExplained>
- Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994

Conclusion

Le pattern Observer et l'Event Bus permettent de concevoir des systèmes réactifs et modulaires.

Ils délèguent la gestion des événements à des mécanismes standardisés, simples à mettre en œuvre et à maintenir.

4-1-2-1

Réactivité des applications avec Observer et Event Bus

Concepts clés de la réactivité

- Réactivité = capacité à répondre rapidement à entrées, changements, erreurs
- Gestion d'événements, flux de données et notifications en temps réel
- Architecture découplée basée sur la propagation d'événements

4-1-2-2

Le pattern Observer

- Modèle pub-sub local
- Un sujet notifie plusieurs observateurs
- Mise à jour automatique et immédiate des états dépendants
- Facilite la propagation d'événements dans une application
- Base pour interfaces utilisateur réactives

4-1-2-3

Event Bus pour les applications réactives

- Extension du pattern Observer (centralisé ou distribué)
- Multiples producteurs et consommateurs d'événements indépendants
- Asynchronisme natif via files d'attente ou brokers
- Gestion avancée : filtrage, priorisation des événements

4-1-2-4

Exemple avec API Reactor (Spring)

```
import reactor.core.publisher.Flux;

public class ReactiveExample {
    public static void main(String[] args) {
        Flux<String> eventBus = Flux.just("evenement1", "evenement2", "evenement3");

        eventBus.subscribe(event -> {
            System.out.println("Réception et traitement de : " + event);
        });

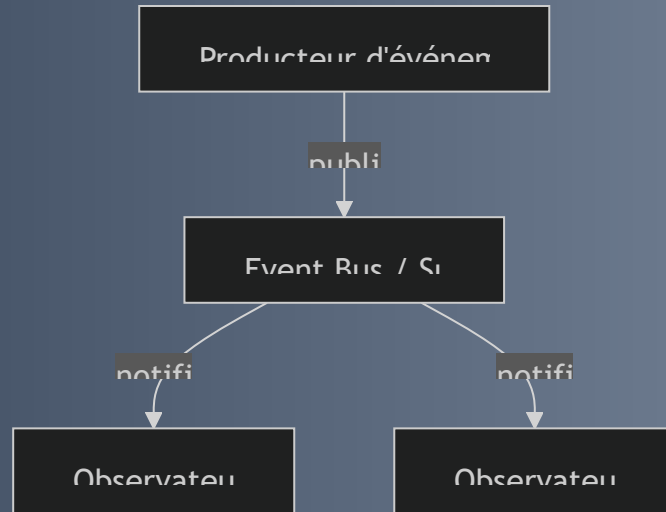
        // Flux terminera automatiquement car il est fini ici
    }
}
```

Sortie :

```
Réception et traitement de : evenement1
Réception et traitement de : evenement2
Réception et traitement de : evenement3
```

4-1-2-5

Modèle réactif : Observer / Event Bus



4-1-2-6

Bénéfices pratiques

- Réactivité utilisateur améliorée : interfaces mises à jour en temps réel
- Meilleure scalabilité : traitement événementiel distribué et asynchrone
- Couplage faible : maintenance et évolution facilitées
- Support du traitement d'erreur et backpressure dans flux complexes

4-1-2-7

Cas d'usage

- Interfaces utilisateur dynamiques (ex : React, Angular avec RxJS)
- Microservices événementiels avec Kafka, RabbitMQ
- Applications IoT et systèmes temps réel

4-1-2-8

Sources utilisées

- Project Reactor documentation – <https://projectreactor.io/docs/core/release/reference/>
- Refactoring Guru, "Observer Pattern" – <https://refactoring.guru/design-patterns/observer>
- Baeldung, "Introduction to Reactive Programming with Reactor" – <https://www.baeldung.com/reactor-core>
- Gamma et al., *Design Patterns*, Addison-Wesley, 1994

Conclusion

Le pattern Observer et le modèle Event Bus facilitent :

- La gestion fluide des événements
 - Une communication efficace et scalable entre composants logiciels
- Fondations clés pour construire des applications réactives et évolutives.

4-2-1-1

Encapsulation d'algorithmes interchangeables avec le pattern Strategy

- Pattern **Strategy** : famille d'algorithmes encapsulés chacun dans une classe.
- Rendre ces algorithmes interchangeables dans un contexte.
- Objectif : isoler le comportement variable du code client pour faciliter extensibilité & maintenance.

4-2-1-2

Principe du pattern Strategy

- Algorithmes encapsulés en objets respectant une interface commune.
- Le contexte contient une référence à une stratégie et délègue la tâche.
- Possibilité de changer l'algorithme à l'exécution sans modifier le contexte.
- Évite les structures conditionnelles complexes (if/else, switch).

4-2-1-3

Exemple : Calcul du coût de livraison avec différentes stratégies

Interface Strategy

```
interface DeliveryStrategy {  
    double calculateCost(double weight);  
}
```

Implémentations concrètes de DeliveryStrategy

```
class CourierDelivery implements DeliveryStrategy {
    @Override
    public double calculateCost(double weight) {
        return weight * 5.0; // Tarif fixe par kilo
    }
}

class DroneDelivery implements DeliveryStrategy {
    @Override
    public double calculateCost(double weight) {
        return weight * 8.0 + 10; // Tarif plus élevé + frais fixe
    }
}

class PostalDelivery implements DeliveryStrategy {
    @Override
    public double calculateCost(double weight) {
        return weight * 3.0 + 2; // Moins cher mais minimum 2€
    }
}
```

Contexte : délégation de la stratégie

```
class DeliveryContext {  
    private DeliveryStrategy strategy;  
  
    public DeliveryContext(DeliveryStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void setStrategy(DeliveryStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void executeStrategy(double weight) {  
        double cost = strategy.calculateCost(weight);  
        System.out.println("Coût de livraison : " + cost + " €");  
    }  
}
```

Utilisation du pattern Strategy

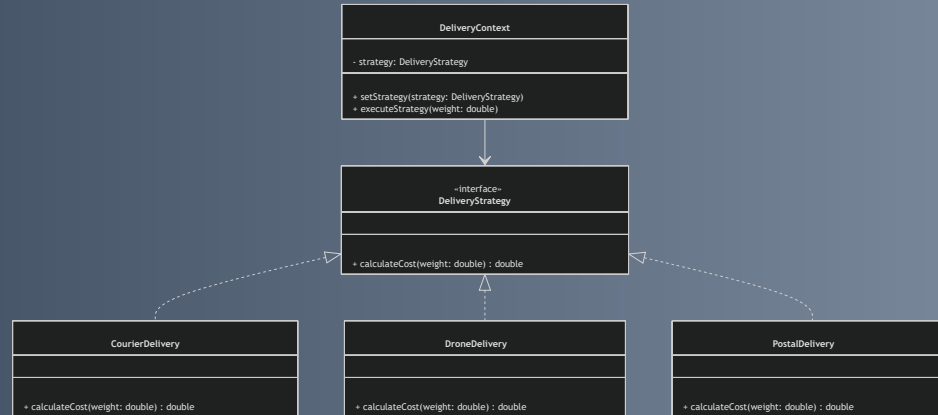
```
public class Client {  
    public static void main(String[] args) {  
        DeliveryContext context = new DeliveryContext(new CourierDelivery());  
        context.executeStrategy(10); // Coût selon Courier  
  
        context.setStrategy(new DroneDelivery());  
        context.executeStrategy(10); // Coût selon Drone  
  
        context.setStrategy(new PostalDelivery());  
        context.executeStrategy(10); // Coût selon Poste  
    }  
}
```

Sortie attendue :

```
Coût de livraison : 50.0 €  
Coût de livraison : 90.0 €  
Coût de livraison : 32.0 €
```

4-2-1-7

Diagramme du pattern Strategy



4-2-1-8

Avantages du pattern Strategy

- Séparation claire entre l'algorithme et son utilisation.
- Extensibilité facilitée : ajout de stratégies sans modifier le contexte.
- Remplacement dynamique de l'algorithme au moment de l'exécution.
- Réduction de la complexité dans les classes clientes.

4-2-1-9

Cas d'usage typiques

- Choix dynamique d'algorithmes (tri, compression, cryptage).
- Gestion flexible des règles de tarifs, promotions ou calculs.
- Plugins ou modules extensibles dans des applications.

4-2-1-10

Sources utilisées

- Refactoring Guru, "Strategy design pattern" — <https://refactoring.guru/design-patterns/strategy>
- Baeldung, "Strategy Pattern in Java" — <https://www.baeldung.com/java-strategy-pattern>
- Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994

Conclusion

Le pattern Strategy s'impose dès que plusieurs variantes de comportement doivent coexister et évoluer indépendamment.
Il améliore la lisibilité et la modularité du code, offrant une forte flexibilité d'utilisation.

4-3-1-1

Encapsulation des actions avec le pattern Command

Introduction

- Le pattern **Command** encapsule une action sous forme d'objet.
- Sépare l'émetteur de la demande de son exécution.
- Facilite le stockage, la transmission, l'annulation et la ré-exécution des commandes.
- Offre une grande flexibilité dans la manipulation des actions.

4-3-1-2

Principe du pattern Command

- Une **commande** contient une action et ses paramètres.
- Elle expose une méthode d'exécution `execute()`.
- Le **client** crée une commande et la donne à un **invoker**.
- Le **receveur** (receiver) réalise l'action concrète.

Bénéfices :

- Découplage clair entre demandeur et exécutant.
- Gère files d'attente, undo/redo, macros facilement.

Exemple simple : contrôle d'un système d'éclairage

Interface Command

```
public interface Command {  
    void execute();  
}
```

Receveur (Light)

```
public class Light {  
    public void on() {  
        System.out.println("Lumière allumée");  
    }  
    public void off() {  
        System.out.println("Lumière éteinte");  
    }  
}
```

4-3-1-4

Exemple : Commandes concrètes

```
public class LightOnCommand implements Command {  
    private Light light;  
    public LightOnCommand(Light light) { this.light = light; }  
    @Override  
    public void execute() { light.on(); }  
}  
  
public class LightOffCommand implements Command {  
    private Light light;  
    public LightOffCommand(Light light) { this.light = light; }  
    @Override  
    public void execute() { light.off(); }  
}
```

4-3-1-5

Exemple : Invoker (Télécommande simplifiée)

```
public class RemoteControl {  
    private Command slot;  
  
    public void setCommand(Command command) {  
        this.slot = command;  
    }  
  
    public void pressButton() {  
        if (slot != null) {  
            slot.execute();  
        }  
    }  
}
```

4-3-1-6

Utilisation

```
public class Client {  
    public static void main(String[] args) {  
        Light livingRoomLight = new Light();  
        Command lightsOn = new LightOnCommand(livingRoomLight);  
        Command lightsOff = new LightOffCommand(livingRoomLight);  
  
        RemoteControl remote = new RemoteControl();  
  
        remote.setCommand(lightsOn);  
        remote.pressButton();  
  
        remote.setCommand(lightsOff);  
    }  
}
```

Sortie :

```
</CodeBlockWrapper>  
Lumière allumée
```

4-3-1-8

Avantages du pattern Command

- **Découplage** entre émetteur et exécuter.
- **Gestion facilitée** des files d'attente, undo/redo et macros.
- **Extensible** : nouvelles commandes sans modifier l'existant.
- **Enregistrement possible** pour exécution différée ou distante.

4-3-1-9

Domaines d'application

- Interfaces graphiques avec undo/redo.
- Scripts et macros dans les applications.
- Traitement différé ou distribué des opérations.
- Implémentations de transactions.

4-3-1-10

Synthèse

Le pattern Command structure les actions en objets autonomes, ce qui permet de gérer aisément leur exécution, report, annulation, et composition, tout en maintenant un code clair, souple et maintenable.

Sources utilisées

- Refactoring Guru, "Command Pattern"
<https://refactoring.guru/design-patterns/command>
- Baeldung, "Command Pattern in Java"
<https://www.baeldung.com/java-command-pattern>
- Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

4-3-2-1

Gestion des undo/redo avec le pattern Command

- Le **pattern Command** encapsule les actions dans des objets.
- Permet de stocker un **historique** des commandes exécutées.
- Facilite les fonctionnalités **undo** (annulation) et **redo** (rétablissement).
- Découple la logique métier de la gestion des commandes.

4-3-2-2

Principes clés du pattern Command pour undo/redo

- Chaque commande propose une méthode :
 - `execute()` pour effectuer l'action.
 - `undo()` pour annuler cette action.
- Les commandes exécutées sont empilées dans une **pile (undoStack)**.
- Les commandes annulées sont stockées dans une autre pile (**redoStack**).
- Nouvelle action → vidage de la pile de redo.

4-3-2-3

Exemple : Interface Command

```
public interface Command {  
    void execute();  
    void undo();  
}
```

4-3-2-4

Exemple : Receveur TextEditor

```
public class TextEditor {  
    private StringBuilder text = new StringBuilder();  
  
    public void addText(String newText) {  
        text.append(newText);  
        System.out.println("Texte actuel : " + text);  
    }  
  
    public void removeText(int length) {  
        int start = text.length() - length;  
        if (start >= 0) {  
            text.delete(start, text.length());  
            System.out.println("Texte après annulation : " + text);  
        }  
    }  
}
```

Exemple : Commande concrète AddTextCommand

```
public class AddTextCommand implements Command {  
    private TextEditor editor;  
    private String textToAdd;  
  
    public AddTextCommand(TextEditor editor, String textToAdd) {  
        this.editor = editor;  
        this.textToAdd = textToAdd;  
    }  
  
    @Override  
    public void execute() {  
        editor.addText(textToAdd);  
    }  
  
    @Override  
    public void undo() {  
        editor.removeText(textToAdd.length());  
    }  
}
```

Exemple : Gestionnaire de commandes (undo/redo)

```
public class CommandManager {  
    private Stack<Command> undoStack = new Stack<>();  
    private Stack<Command> redoStack = new Stack<>();  
  
    public void executeCommand(Command command) {  
        command.execute();  
        undoStack.push(command);  
        redoStack.clear();  
    }  
  
    public void undo() {  
        if (!undoStack.isEmpty()) {  
            Command cmd = undoStack.pop();  
            cmd.undo();  
            redoStack.push(cmd);  
        }  
    }  
  
    public void redo() {  
        if (!redoStack.isEmpty()) {  
            Command cmd = redoStack.pop();  
            cmd.execute();  
            undoStack.push(cmd);  
        }  
    }  
}
```


Utilisation client

```
public class Client {  
    public static void main(String[] args) {  
        TextEditor editor = new TextEditor();  
        CommandManager manager = new CommandManager();  
  
        Command cmd1 = new AddTextCommand(editor, "Bonjour ");  
        Command cmd2 = new AddTextCommand(editor, "le monde!");  
  
        manager.executeCommand(cmd1);  
        manager.executeCommand(cmd2);  
  
        manager.undo();  
        manager.redo();  
    }  
}
```

Sortie attendue :

```
Texte actuel : Bonjour  
Texte actuel : Bonjour le monde!
```

4-3-2-9

Avantages du pattern Command pour undo/redo

- **Indépendance** des commandes vis-à-vis du client.
- **Historique des actions** fiable grâce aux piles undo/redo.
- **Extensibilité** simple en ajoutant de nouvelles commandes.
- **Centralisation** de la logique d'annulation et ré-exécution.
- Facilite la **maintenance** et l'évolution du code.

4-3-2-10

Domaines d'application fréquents

- Éditeurs de texte, graphiques, vidéo.
- Applications bureautiques avec modifications utilisateurs.
- Jeux vidéo (gestion d'états et d'actions).
- Systèmes transactionnels avec besoin de rollback.

4-3-2-11

Sources utilisées

- Refactoring Guru, "Command pattern"
- Baeldung, "Command Pattern in Java"
- Microsoft Docs, "Undo-Redo with Command pattern"
- Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994

Conclusion

Le pattern Command, enrichi avec `undo()`, fournit un cadre propre pour implémenter undo/redo.

Il permet un code modulaire, facile à maintenir, et indispensable dans les logiciels interactifs.

Cette approche garantit un historique fiable et une grande flexibilité dans la gestion des actions utilisateur.

4-4-1-1

Définition du squelette d'un algorithme avec le pattern Template Method

Introduction

- Le pattern **Template Method** définit la structure globale d'un algorithme dans une méthode "template".
- Permet de **réutiliser** et d'assurer la **cohérence** des processus.
- Offre la **flexibilité** pour personnaliser certaines étapes dans des sous-classes.

slide: 4-4-1-3

Principe du pattern Template Method

- Une **classe abstraite** définit une méthode template : le **squelette** de l'algorithme.
- Certaines étapes sont implémentées dans la classe abstraite, d'autres sont **abstraites**.
- Les sous-classes fournissent les implémentations spécifiques des étapes abstraites.
- Permet de **fixer l'ordre d'exécution** tout en adaptant les détails.

Exemple : préparation d'une boisson chaude

Classe abstraite Beverage

```
public abstract class Beverage {  
    // Template method  
    public final void prepareRecipe() {  
        boilWater();  
        brew();  
        pourInCup();  
        addCondiments();  
    }  
  
    private void boilWater() {  
        System.out.println("Faire bouillir de l'eau");  
    }  
  
    protected abstract void brew();  
  
    private void pourInCup() {  
        System.out.println("Verser dans la tasse");  
    }  
}
```

Exemple (suite) : Sous-classes concrètes

```
public class Tea extends Beverage {
    @Override
    protected void brew() {
        System.out.println("Infuser le thé");
    }

    @Override
    protected void addCondiments() {
        System.out.println("Ajouter du citron");
    }
}

public class Coffee extends Beverage {
    @Override
    protected void brew() {
        System.out.println("Préparer le café moulu");
    }

    @Override
    protected void addCondiments() {
        System.out.println("Ajouter du sucre et du lait");
    }
}
```


slide: 4-4-1-6

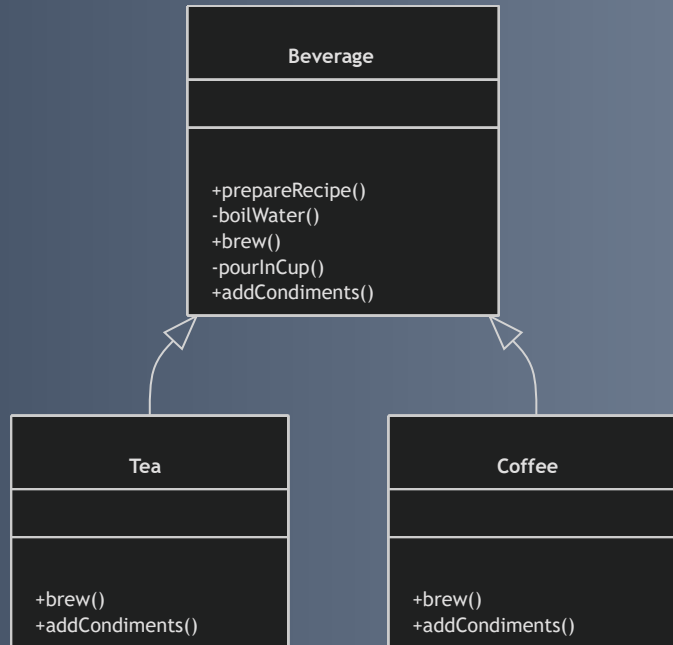
Utilisation et sortie attendue

```
public class Client {  
    public static void main(String[] args) {  
        Beverage tea = new Tea();  
        Beverage coffee = new Coffee();  
  
        System.out.println("Préparation du thé:");  
        tea.prepareRecipe();  
  
        System.out.println("\nPréparation du café:");  
        coffee.prepareRecipe();  
    }  
}
```

Sortie console :

```
Préparation du thé:  
Faire bouillir de l'eau  
Infuser le thé  
Verser dans la tasse  
Ajouter du citron  
  
Préparation du café:  
Faire bouillir de l'eau  
Préparer le café moulu  
Verser dans la tasse  
Ajouter du sucre et du lait
```

Diagramme du pattern Template Method



slide: 4-4-1-8

Avantages du pattern Template Method

- **Réutilisation** du code commun dans la classe abstraite.
- **Encapsulation du flux d'algorithme**, contrôle strict de la séquence.
- **Flexibilité** grâce à la personnalisation des étapes dans les sous-classes.
- Facilite la **maintenance** et l'extension des algorithmes.

slide: 4-4-1-9

Domaines d'application

- Frameworks et bibliothèques avec processus standard à adapter localement.
- Traitement de données multi-étapes (validation, transformation, stockage).
- Interfaces graphiques combinant étapes communes et comportements spécifiques.

Sources utilisées

- Refactoring Guru, "Template Method pattern", <https://refactoring.guru/design-patterns/template-method>
- Baeldung, "Template Method in Java", <https://www.baeldung.com/java-template-method-pattern>
- Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

Conclusion

Le pattern Template Method dissocie la **partie commune** et la **partie variable** dans un algorithme.

Il garantit l'ordre d'exécution tout en favorisant :

- L'**adaptabilité**
- La **simplicité**
- La **robustesse** du code.

4-4-2-1

Spécialisation des étapes de l'algorithme avec le pattern Template Method

- Pattern **Template Method** :
Définit un algorithme dans une méthode abstraite.
- Sous-classes spécialisent certaines étapes :
 - Obligatoire via méthodes abstraites
 - Optionnelle via *hooks* (accroches)

4-4-2-2

Mécanismes de spécialisation dans Template Method

1. Méthodes abstraites

1. Imposent aux sous-classes d'implémenter des étapes spécifiques
2. Assurent un algorithme complet et fonctionnel

2. Hooks (accroches)

1. Implémentation par défaut dans la classe abstraite
2. Redéfinition facultative par sous-classes pour modifier le comportement
3. Exemple : exécuter ou ignorer une étape

Exemple concret : préparation d'une boisson avec hook optionnel

Classe abstraite Beverage définit la recette (template method) :

```
public abstract class Beverage {  
    public final void prepareRecipe() {  
        boilWater();  
        brew();  
        pourInCup();  
        if (customerWantsCondiments()) {  
            addCondiments();  
        }  
    }  
  
    private void boilWater() { System.out.println("Faire bouillir de l'eau"); }  
    protected abstract void brew();  
    private void pourInCup() { System.out.println("Verser dans la tasse"); }  
    protected abstract void addCondiments();  
  
    protected boolean customerWantsCondiments() { return true; }  
}
```

4-4-2-4

Sous-classes spécialisant avec méthodes abstraites et hook

```
public class Coffee extends Beverage {
    @Override
    protected void brew() { System.out.println("Préparer le café moulu"); }
    @Override
    protected void addCondiments() { System.out.println("Ajouter du sucre et du lait"); }
}

public class Tea extends Beverage {
    @Override
    protected void brew() { System.out.println("Infuser le thé"); }
    @Override
    protected void addCondiments() { System.out.println("Ajouter du citron"); }
    @Override
    protected boolean customerWantsCondiments() { return false; } // hook modifié
}
```

4-4-2-5

Test d'utilisation et sortie attendue

```
public class Client {  
    public static void main(String[] args) {  
        Beverage coffee = new Coffee();  
        Beverage tea = new Tea();  
  
        System.out.println("Préparation du café:");  
        coffee.prepareRecipe();  
  
        System.out.println("\nPréparation du thé:");  
        tea.prepareRecipe();  
    }  
}
```

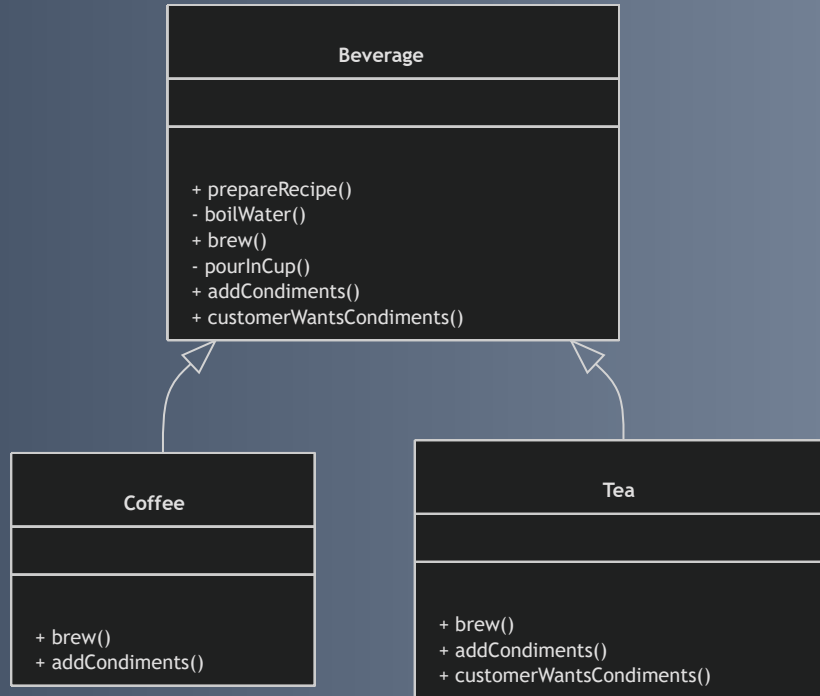
Sortie :

```
Préparation du café:  
Faire bouillir de l'eau  
Préparer le café moulu  
Verser dans la tasse  
Ajouter du sucre et du lait
```

```
Préparation du thé:  
Faire bouillir de l'eau  
Infuser le thé  
Verser dans la tasse
```

Le thé est servi sans condiments grâce à la redéfinition du hook.

Diagramme de classes



4-4-2-7

Points clés à retenir

- **Méthodes abstraites** : étapes obligatoires, personnalisables par sous-classes
- **Hooks (accroches)** : étapes optionnelles, comportement par défaut modifiable
- Assure une structure claire et flexible d'algorithme
- Facilite maintenance et évolution des algorithmes complexes

4-4-2-8

Sources

- Refactoring Guru, "Template Method pattern", <https://refactoring.guru/design-patterns/template-method>
- Baeldung, "Template Method in Java", <https://www.baeldung.com/java-template-method-pattern>
- Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994

Conclusion

La spécialisation des étapes dans le pattern Template Method,
à travers méthodes abstraites et hooks optionnels,
permet d'adapter finement un algorithme défini
tout en assurant robustesse et maintenabilité du code.

4-5-1-1

Parcours d'éléments avec Iterator et Chain of Responsibility

- Parcours fréquent en programmation sur collections ou chaînes de traitement
- Complexité liée à la structure interne ou au chemin de traitement variable
- Deux patterns clés : **Iterator** et **Chain of Responsibility**
- Objectif : Séparer les mécanismes de parcours et de traitement pour plus de flexibilité

4-5-1-2

Le pattern Iterator

Principe

- Accès séquentiel aux éléments d'une collection sans exposer sa structure interne
- Uniformise le parcours quel que soit le type (listes, arbres, tables de hachage)

Interface de base

```
public interface Iterator<T> {  
    boolean hasNext();  
    T next();  
}
```

Iterator - Exemple simplifié

```
public class NameRepository {  
    private String[] names = {"Alice", "Bob", "Charlie"};  
  
    public Iterator<String> getIterator() {  
        return new NameIterator();  
    }  
  
    private class NameIterator implements Iterator<String> {  
        int index;  
  
        public boolean hasNext() {  
            return index < names.length;  
        }  
  
        public String next() {  
            if(hasNext()) {  
                return names[index++];  
            }  
            return null;  
        }  
    }  
}
```

Utilisation :

```
Iterator<String> iterator = new NameRepository().getIterator();  
while(iterator.hasNext()) {  
    System.out.println(iterator.next());  
}
```

<CodeBlockWrapper v-bind="{}" :title="" :ranges='[]'>

Sortie :

```
</CodeBlockWrapper>  
Alice  
Bob  
Charlie
```

4-5-1-4

Le pattern Chain of Responsibility

Principe

- Requête envoyée à une chaîne d'objets (handlers)
- Chaque objet décide de traiter ou passer au suivant
- Diminue le couplage entre émetteur et récepteur

4-5-1-5

Chain of Responsibility

```
abstract class Handler {  
    protected Handler next;  
  
    public void setNext(Handler next) {  
        this.next = next;  
    }  
  
    public abstract void handleRequest(String request);  
}
```

--> Suite -->

```
class ConcreteHandlerA extends Handler {
    public void handleRequest(String request) {
        if (request.equals("A")) {
            System.out.println("Handled by A");
        } else if (next != null) {
            next.handleRequest(request);
        }
    }
}
```

```
class ConcreteHandlerB extends Handler {
    public void handleRequest(String request) {
        if (request.equals("B")) {
            System.out.println("Handled by B");
        } else if (next != null) {
            next.handleRequest(request);
        }
    }
}
```

Usage & sorties :

```
handlerA.setNext(handlerB);  
handlerA.handleRequest("B"); // Handled by B  
handlerA.handleRequest("A"); // Handled by A  
handlerA.handleRequest("C"); // Non traité
```

4-5-1-7

Comparaison et complémentarité

Aspect	Iterator	Chain of Responsibility
But	Parcourir séquentiellement une collection sans exposer sa structure	Passer une requête à une chaîne jusqu'au traitement
Structure	Itérateur avec <code>hasNext()</code> , <code>next()</code>	Chaîne d'objets liés avec <code>setNext()</code> , <code>handleRequest()</code>
Utilisation	Parcours homogène de données (listes, arbres)	Décision dynamique sur le handler qui traite la requête
Couplage	Détache collection et mécanisme de parcours	Détache émetteur et récepteur concret

4-5-1-8

Applications pratiques

- **Iterator**
 - Parcourir des collections personnalisées
 - Accès uniforme à différents types de données (frameworks, API)
- **Chain of Responsibility**
 - Traitement d'événements dans interfaces graphiques
 - Gestion successive des requêtes web
 - Validation en plusieurs étapes, journalisation (logging)

4-5-1-9

Sources utilisées

- Refactoring Guru, "Iterator Pattern"
- Refactoring Guru, "Chain of Responsibility Pattern"
- Baeldung, "Java Design Patterns – Iterator and Chain of Responsibility"
- Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994

Conclusion

- Iterator et Chain of Responsibility traitent du parcours et traitement des éléments ou requêtes
- Ils offrent modularité, flexibilité et séparation des responsabilités
- Fondamentaux pour manipuler collections ou flux de requêtes dans du code maintenable et évolutif

4-5-2-1

Délégation de responsabilité entre objets

Patterns Iterator et Chain of Responsibility

- Délégation : mécanisme clé en conception orientée objet
- **Iterator** : délégation du parcours d'une collection
- **Chain of Responsibility** : délégation du traitement d'une requête entre objets
- Objectif : concevoir des systèmes souples, extensibles, découplés

4-5-2-2

Délégation dans le pattern Iterator – Concept

- Le parcours d'une collection est externalisé à un **itérateur**
- La collection ne révèle pas sa structure interne
- L'itérateur dispose de méthodes standardisées :
 - `hasNext()` : vérifie la présence d'un élément suivant
 - `next()` : récupère l'élément courant et avance la position
- Le client utilise l'itérateur pour naviguer dans la collection

Iterator – Exemple simplifié

```
public interface Iterator<T> {  
    boolean hasNext();  
    T next();  
}  
  
public class ListIterator<T> implements Iterator<T> {  
    private List<T> list;  
    private int position = 0;  
  
    public ListIterator(List<T> list) {  
        this.list = list;  
    }  
  
    public boolean hasNext() {  
        return position < list.size();  
    }  
  
    public T next() {  
        if (!hasNext()) throw new NoSuchElementException();  
        return list.get(position++);  
    }  
}
```

```
public class Client {  
    public static void main(String[] args) {  
        List<String> names = Arrays.asList("Ana", "Ben", "Cara");  
        Iterator<String> iterator = new ListIterator<>(names);  
  
        while (iterator.hasNext()) {  
            System.out.println(iterator.next());  
        }  
    }  
}
```

4-5-2-4

Iterator – Avantage clé

- La collection **délègue** la navigation à l'itérateur
- Modification de la structure interne possible sans impacter le client
- Permet une itération indépendante de la structure sous-jacente

4-5-2-5

Délégation dans le pattern Chain of Responsibility – Concept

- Chaîne d'objets (handlers) liés entre eux
- Chaque handler décide s'il peut traiter la requête
- Sinon, il **délègue** la requête au handler suivant dans la chaîne
- La requête circule jusqu'à un handler capable de la traiter

Chain of Responsibility – Exemple simplifié

```
abstract class Handler {
    protected Handler next;

    public void setNext(Handler next) {
        this.next = next;
    }

    public void handle(String request) {
        if (canHandle(request)) {
            process(request);
        } else if (next != null) {
            next.handle(request);
        }
    }

    protected abstract boolean canHandle(String request);
    protected abstract void process(String request);
}
```

```
class AuthenticationHandler extends Handler {  
    protected boolean canHandle(String request) {  
        return request.equals("auth");  
    }  
  
    protected void process(String request) {  
        System.out.println("Authentification traitée");  
    }  
}  
  
class LoggingHandler extends Handler {  
    protected boolean canHandle(String request) {  
        return request.equals("log");  
    }  
  
    protected void process(String request) {  
        System.out.println("Logging effectué");  
    }  
}
```

--> Suite -->

```
public class Client {  
    public static void main(String[] args) {  
        Handler auth = new AuthenticationHandler();  
        Handler log = new LoggingHandler();  
        auth.setNext(log);  
  
        auth.handle("auth"); // Authentification traitée  
        auth.handle("log");  // Logging effectué  
        auth.handle("other"); // Non traité (aucun handler)  
    }  
}
```

4-5-2-7

Chain of Responsibility – Avantage clé

- Chaque handler **délègue** le traitement s'il ne peut pas gérer la requête
- Création d'une chaîne d'objets modulable et dynamique
- Facilite la composition flexible des traitements

4-5-2-9

Synthèse et recommandations

Pattern	Objet délégué	Type de responsabilité déléguée	Flexibilité apportée
Iterator	Itérateur	Parcours d'éléments	Itération indépendante de la structure d'origine
Chain of Responsibility	Handler suivant dans la chaîne	Traitement ou gestion d'une requête	Composition dynamique d'objets pour le traitement

- Gestion de collections : privilégier **Iterator**
- Traitements chaînés ou multi-étapes : privilégier **Chain of Responsibility**

Sources utilisées

- Refactoring Guru, *Iterator Pattern*
<https://refactoring.guru/design-patterns/iterator>
- Refactoring Guru, *Chain of Responsibility Pattern*
<https://refactoring.guru/design-patterns/chain-of-responsibility>
- Baeldung, *Chain of Responsibility en Java*
<https://www.baeldung.com/java-chain-of-responsibility-pattern>
- Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994

Conclusion

- La **délégation** est un mécanisme fondamental en conception objet
- Iterator et Chain of Responsibility illustrent deux usages clés :
 - Délégation du parcours d'une collection
 - Délégation du traitement d'une requête entre objets
- Ces patterns améliorent la flexibilité et la maintenabilité des systèmes complexes